

COMPUTER GRAPHICS & MULTIMEDIA

Complete Question Bank with Full Solutions



1. Introduction to Computer Graphics & Applications

Q1. Explain the need and use of graphics in the field of IT. [8 Marks]

In Simple English

Pictures explain things faster than plain text or numbers. In IT, graphics let people see data, designs, and simulations visually, which makes complex information much easier to understand and use.

Technical Explanation

Computer graphics provides the tools to create, manipulate, and display visual images using a computer, and has become essential across nearly every branch of Information Technology because visual representation communicates information far more efficiently than raw text or numeric data.

Need and Use of Graphics in IT

- **Data Visualization:** Charts, graphs, and dashboards let analysts and managers understand large/complex datasets at a glance (business intelligence, scientific visualization).
- **User Interfaces (GUI):** Modern operating systems, apps, and websites rely entirely on graphical icons, windows, and menus for intuitive interaction.
- **CAD/CAM:** Engineers and architects design machine parts, circuits, and buildings using computer-aided design tools.
- **Entertainment & Gaming:** Movies, animation, and video games depend on real-time and rendered graphics for realism.
- **Education & Training:** Simulators (flight simulators, medical training) use graphics to safely recreate real-world scenarios.
- **Simulation & Modeling:** Weather forecasting, molecular modeling, and physical simulations rely on graphical rendering to interpret results.

Q2. Define Computer Graphics. [8 Marks]**In Simple English**

Computer graphics is simply the art and science of making pictures with a computer.

Technical Explanation

Computer Graphics is a field of computer science concerned with generating, manipulating, and representing visual images (pictures, diagrams, animations) digitally using computer hardware and software. It encompasses algorithms for drawing shapes, applying color and texture, transforming objects, and rendering realistic 2D/3D scenes on a display device.

Q3. Define the terms computer graphics, interactive graphics, and non-interactive graphics.

[8 Marks]

In Simple English

Interactive graphics respond immediately when you do something (like a video game reacting to your button press). Non-interactive graphics are just played or displayed without you being able to change them, like a pre-rendered movie.

Technical Explanation

Computer Graphics: The field concerned with creating, storing, and manipulating images/models using a computer.

Interactive Graphics: A graphics system where the user can control and modify the picture in real time through input devices (mouse, keyboard, joystick), with immediate visual feedback. Example: video games, CAD tools, flight simulators.

Non-Interactive Graphics: Also called passive graphics — the user has no control over the image once it is generated; the picture is simply a fixed output. Example: a pre-rendered movie, a static chart printed on paper, television broadcast.

Q4. What is the difference between computer graphics and image processing? [8 Marks]

In Simple English

Computer graphics creates pictures out of nothing (from mathematical descriptions). Image processing takes a picture that already exists (like a photo) and improves or analyzes it.

Technical Explanation

Aspect	Computer Graphics	Image Processing
Input	Geometric/mathematical description of objects (models, coordinates)	An existing image (e.g., a photograph or scanned picture)
Output	A newly synthesized image/picture	An enhanced, corrected, or analyzed version of the input image
Process Direction	Model/description → Image (synthesis)	Image → Image or Image → Description (analysis)
Example	Rendering a 3D animated character	Removing noise from a photo, edge detection, medical image enhancement

Q5. Explain the application of computer graphics. [8 Marks]

In Simple English

Computer graphics is used almost everywhere images appear on a screen — games, movies, medical scans, engineering designs, weather maps, and more.

Major Application Areas

- **Presentation Graphics:** Bar charts, pie charts, and graphs to summarize statistical/business data for reports and presentations.
- **CAD (Computer-Aided Design):** Designing buildings, cars, circuit boards, and mechanical parts with precision.
- **Entertainment:** Movies (VFX/CGI), television, and video games use computer-generated visuals and animation.
- **Education & Training:** Simulators for pilots, surgeons, and drivers use graphics to safely simulate real environments.
- **Visualization:** Scientific, engineering, and business data is visualized to reveal patterns not obvious in raw numbers.
- **Image Processing:** Enhancing/analyzing photographs, medical scans (X-ray, MRI, CT), and satellite images.
- **Graphical User Interfaces (GUI):** Icons, windows, menus for operating systems and applications.
- **Art and Commercial Design:** Digital painting, logo design, advertising graphics.

Q6. Explain the application of computer graphics in Visualization and Image Processing in detail. [8 Marks]

In Simple English

Visualization turns raw numbers into pictures so we can spot patterns at a glance. Image processing takes a real picture and cleans it up or extracts useful information from it.

Visualization

Visualization is the use of computer graphics to create visual representations of abstract, often large and complex, numerical data — helping researchers, scientists, and analysts to interpret trends, patterns, and relationships that would be difficult to detect from raw numbers alone.

- Scientific Visualization: Visualizing simulation results (fluid flow, weather patterns, molecular structures)
- Business Visualization: Dashboards, trend graphs, and charts for decision-making
- Medical Visualization: 3D reconstruction of organs from CT/MRI scan slices

Image Processing

Image processing applies computational techniques to an existing image in order to improve its quality or extract meaningful information from it. Unlike graphics (which synthesizes new images), image processing works on images that already exist.

- Image Enhancement: Improving contrast, sharpness, brightness of photographs
- Image Restoration: Removing noise, blur, or scratches from old/damaged photographs
- Pattern Recognition: Detecting faces, objects, fingerprints, or handwriting within an image
- Medical Imaging: Enhancing X-ray/CT/MRI scans for clearer diagnosis

Q7. How has the evolution of computer graphics technology impacted the gaming industry in terms of realism and player immersion? [8 Marks]

In Simple English

As graphics hardware and techniques got better and better, games went from simple blocky shapes to almost photo-realistic worlds, making players feel like they are really inside the game.

Technical Explanation

- **From 2D Sprites to 3D Polygon Models:** Early games used flat 2D sprites; modern GPUs render millions of textured, shaded 3D polygons per frame.
- **Advanced Shading & Lighting:** Techniques like Phong shading, ray tracing, and global illumination create realistic lighting, shadows, and reflections.
- **Higher Resolutions & Frame Rates:** 4K resolution and 60-144 fps rendering give smoother, sharper motion, reducing the perception of a 'digital' image.
- **Physics Simulation:** Real-time physics engines simulate realistic motion, collisions, and cloth/fluid behavior.
- **VR/AR Integration:** Stereoscopic 3D rendering and head-tracking create a fully immersive first-person experience.

Overall, better graphics hardware (GPUs), rendering algorithms, and display technology together have progressively closed the gap between virtual worlds and reality, dramatically increasing player immersion.

Q8. Differentiate between 2D and 3D graphics. [8 Marks]

In Simple English

2D graphics are flat, like a drawing on paper — only width and height. 3D graphics have depth too, like a real object you could walk around.

Technical Explanation

Aspect	2D Graphics	3D Graphics
Dimensions	Two (x, y)	Three (x, y, z)
Representation	Flat shapes, no depth	Solid objects with depth, volume
Realism	Less realistic	More realistic — supports lighting, shading, perspective
Processing Cost	Low — simpler to compute	High — requires transformation, projection, hidden-surface removal, shading
Examples	Logos, icons, 2D games, diagrams	3D games, animated films, CAD models, VR

Q9 & Q10. In graphics, which dimension is more applicable, and why? [8 Marks]

In Simple English

It depends on the purpose. For simple things like icons or diagrams, 2D is enough and faster. But for realistic simulations, games, and modeling real objects, 3D is far more useful because the real world has depth.

Technical Explanation

Neither is universally 'better' — the appropriate dimensionality depends on the application's requirements:

- **2D is more applicable when:** the application only needs flat representation — user interfaces, business charts, technical diagrams, 2D games — since it requires less processing power and simpler algorithms.
- **3D is more applicable when:** realism, depth perception, and spatial relationships matter — engineering/architectural modeling, medical imaging, films, modern video games, and virtual/augmented reality — because it can represent real-world objects and scenes far more accurately.

In current industry trends, 3D graphics is increasingly more widely applicable because modern hardware (GPUs) can handle its higher computational demand, and most real-world applications (simulation, gaming, VR, digital twins) require the added realism and spatial accuracy that only three dimensions can provide.

2. Display Devices & Hardware

Q1. Define CRT display. [8 Marks]

In Simple English

A CRT (Cathode Ray Tube) is the old, boxy TV/monitor type that draws pictures by firing a beam of electrons at a glowing screen.

Technical Explanation

A Cathode Ray Tube (CRT) is a vacuum-tube display device in which an electron gun emits a focused beam of electrons that is accelerated and deflected (using deflection plates or magnetic coils) to strike specific positions on a phosphor-coated screen, causing the phosphor to glow and produce an image. The intensity of the beam controls brightness, and the beam sweeps rapidly to build up the full picture (refreshing many times per second).

Q2. Compare and contrast beam penetration method and shadow mask method. [8 Marks]

In Simple English

Both are ways to produce color on a CRT screen. Beam penetration uses beam speed to pick a color from layered phosphors; shadow mask uses a metal sheet with tiny holes to make sure each of the three color guns only hits its own matching color dot.

Technical Explanation

Beam-Penetration Method: Used mainly in random-scan (vector) CRT monitors. The screen is coated with two phosphor layers — red (inner) and green (outer). A high-speed electron beam penetrates through to the outer green layer, while a low-speed beam is stopped in the inner red layer. Intermediate speeds produce intermediate colors (orange, yellow) by exciting both layers to different extents. Only 4 colors are typically achievable.

Shadow-Mask Method: Used in raster-scan color CRT monitors, and provides a much wider, more accurate color range. It uses three electron guns (for Red, Green, Blue) and a metal shadow mask perforated with tiny holes positioned precisely in front of a screen coated with clusters (triads) of red, green, and blue phosphor dots. The mask ensures each gun's beam only hits its matching-color dot.

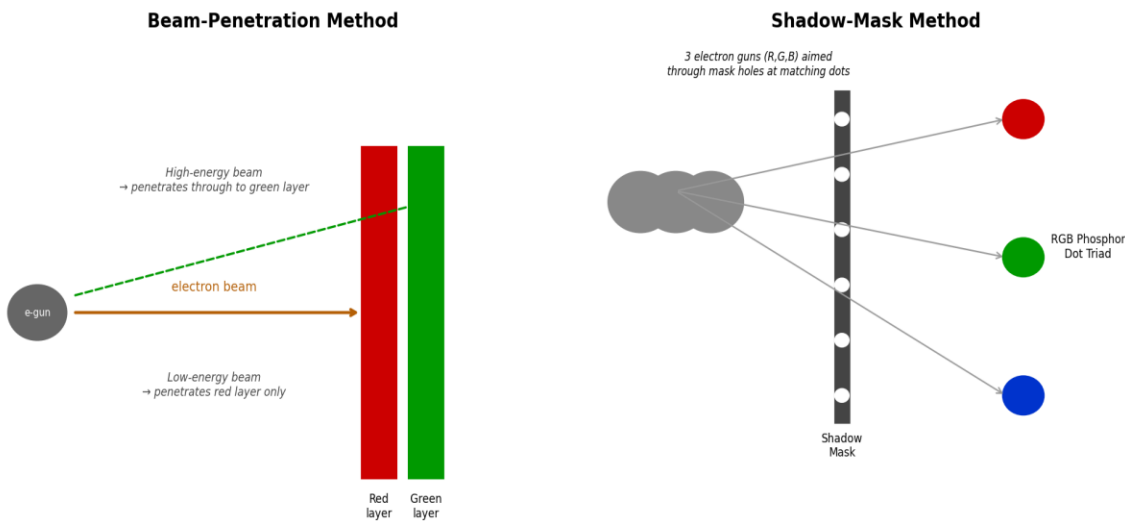


Figure: Beam-Penetration Method vs Shadow-Mask Method

Aspect	Beam-Penetration	Shadow-Mask
Used In	Random-scan (vector) monitors	Raster-scan monitors
Electron Guns	1 (varies beam acceleration/speed)	3 (Red, Green, Blue)
Color Range	Limited (~4 colors)	Wide range (millions of colors)
Mechanism	Beam penetrates phosphor layers based on speed	Mask aligns each gun's beam onto its matching-color phosphor dot
Picture Quality	Lower resolution/quality	Higher resolution and color accuracy

Q3. List out the merits and demerits of the (beam) penetration technique. [8 Marks]

Merits

- Simple in construction — requires only a single electron gun
- Inexpensive compared to shadow-mask systems
- Works well with random/vector-scan systems for simple line drawings

Demerits

- Limited to only about 4 colors (red, green, and two intermediate blends)
- Poor color quality and accuracy compared to shadow-mask (RGB) systems
- Picture quality/resolution is generally lower
- Not suitable for displaying complex, photorealistic, or full-color images

Q4 & Q5. Explain the color generation technique in a CRT monitor. How is color displayed on a monitor? [8 Marks]

In Simple English

Every color you see on a screen is actually made by mixing tiny dots of just three colors — red, green, and blue light — in different amounts.

Technical Explanation

Color CRT monitors generate color using the additive RGB color model, primarily through the shadow-mask technique (or beam-penetration for vector displays):

- The screen is coated with clusters of three phosphor dots — Red, Green, and Blue (an RGB triad) — for every pixel position
 - Three separate electron guns fire beams calibrated to strike only their corresponding color dot, aligned precisely using the shadow mask
 - By varying the intensity of each of the three beams independently, each phosphor dot glows with a different brightness
 - Because the three dots are so close together, the human eye perceptually blends their light into a single combined color (additive color mixing) — e.g., full-intensity Red + Green + Blue produces white light
-

Q6. Define frame buffer. [8 Marks]

In Simple English

A frame buffer is the part of memory that stores exactly what should currently be shown on the screen, pixel by pixel.

Technical Explanation

A frame buffer is a dedicated area of graphics memory (VRAM) that stores the color/intensity value for every pixel of the display. The video controller continuously reads the frame buffer and refreshes the screen (e.g., 60-75 times per second), converting the stored digital pixel values into the analog signals that drive the display. Any change to the displayed image is achieved by writing new values into the frame buffer.

Q7 & Q8. Differentiate / Compare and contrast Random-Scan and Raster-Scan display systems. [8 Marks]

In Simple English

Raster scan draws the whole screen row by row, like reading a book line by line. Random scan only draws the actual lines needed, jumping the beam directly from one endpoint to another, like a pen lifting and moving to draw only specific strokes.

Technical Explanation

Aspect	Raster Scan	Random Scan (Vector)
Method	Electron beam sweeps the entire screen row by row (left-to-right, top-to-bottom)	Beam is directed only to the specific points/lines that form the picture
Storage	Uses a frame buffer to store intensity of every pixel	Uses a display file / refresh list of line-drawing commands
Resolution	Limited by number of scan lines/pixels	Very high resolution, smooth lines
Realism	Supports solid/filled areas, shading, realistic images	Best for line drawings; poor for filled/shaded images
Refresh Rate	Independent of picture complexity	Depends on the number of lines in the picture
Example Devices	Modern TVs, LCD/LED monitors	Older vector graphics terminals, oscilloscopes

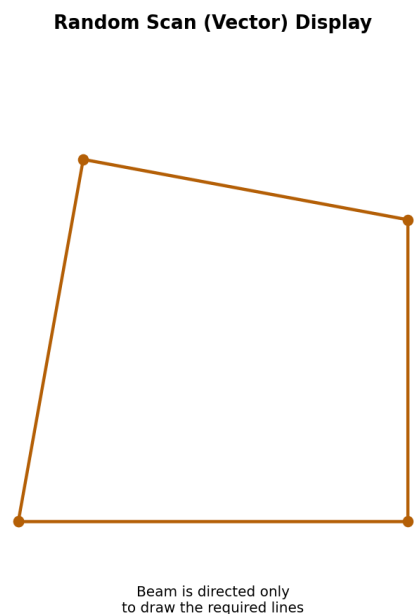
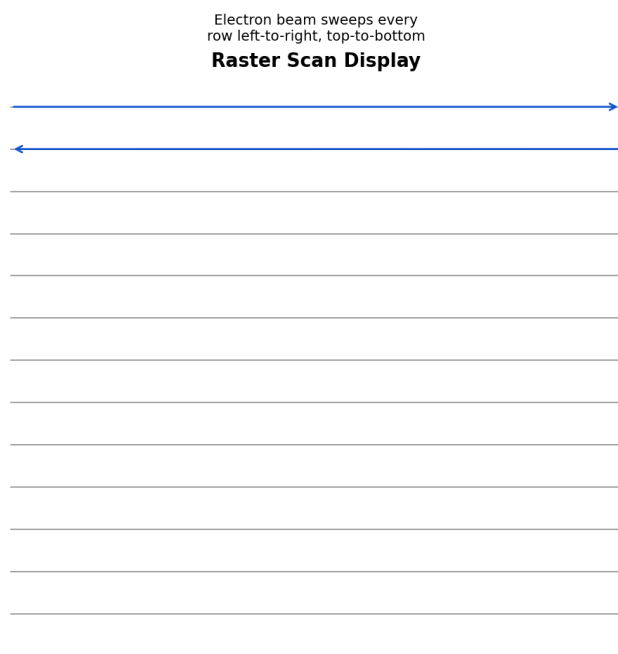


Figure: Raster Scan vs Random (Vector) Scan

Q9. Define Aspect ratio. [8 Marks]

In Simple English

Aspect ratio is just the shape of the screen — how wide it is compared to how tall it is.

Technical Explanation

Aspect ratio is defined as the ratio of vertical points (rows) to horizontal points (columns) — or equivalently the ratio of width to height of the display — that must be maintained so that pictures are displayed without geometric distortion. For instance, a common aspect ratio of 4:3 or 16:9 ensures that a circle drawn on screen still appears as a circle and not an ellipse.

Q10. Numerical: Pixels accessed per second and access time for two raster systems refreshing at 75 fps. [8 Marks]

In Simple English

We just multiply the total number of pixels on screen by how many times per second the screen redraws, to find out how many pixels the system must handle every second — then flip that number to find the time allowed per pixel.

Given

- System 1 resolution: 640×480 , refresh rate = 75 frames/second
- System 2 resolution: 1024×600 , refresh rate = 75 frames/second

System 1: 640×480 @ 75 Hz

Total pixels per frame = $640 \times 480 = 307,200$ pixels
Pixels accessed per second = $307,200 \times 75 = 23,040,000$ pixels/sec
 $= 2.304 \times 10^7$ pixels/sec

Access time per pixel = $1 / (23,040,000)$ seconds
 $\approx 4.34 \times 10^{-8}$ seconds
 ≈ 43.4 nanoseconds (ns)

System 2: 1024×600 @ 75 Hz

Total pixels per frame = $1024 \times 600 = 614,400$ pixels
Pixels accessed per second = $614,400 \times 75 = 46,080,000$ pixels/sec
 $= 4.608 \times 10^7$ pixels/sec

Access time per pixel = $1 / (46,080,000)$ seconds
 $\approx 2.17 \times 10^{-8}$ seconds
 ≈ 21.7 nanoseconds (ns)

System	Resolution	Pixels/Frame	Pixels/Second (@75fps)	Access Time/Pixel
1	640×480	307,200	23,040,000	≈ 43.4 ns
2	1024×600	614,400	46,080,000	≈ 21.7 ns

Note: the higher-resolution system (1024×600) must access roughly twice as many pixels per second, so the display controller has proportionally less time available (about half) to access each individual pixel.

3. Line & Circle Drawing Algorithms

Q1, Q2 & Q3. What is DDA? Describe/Derive the DDA algorithm with points (x1,y1) and (x2,y2). [8 Marks]

In Simple English

A screen is made of a grid of pixels, but a mathematical line is perfectly smooth. DDA (Digital Differential Analyzer) is a simple step-by-step method that picks the closest pixel at each small step along a line, so it looks like a straight line on screen.

Technical Explanation

DDA is an incremental scan-conversion algorithm that computes pixel positions along a line by using the line's slope to determine unit increments in x or y, then rounding to the nearest integer pixel at each step.

Derivation

Given the endpoints (x1, y1) and (x2, y2), the slope of the line is:

$$m = (y_2 - y_1) / (x_2 - x_1) = \Delta y / \Delta x$$

This comes from the general line equation $y = mx + c$. Differentiating with respect to x: $dy/dx = m$. For a unit step in x ($\Delta x = 1$), the corresponding change in y is $\Delta y = m$. Similarly, for a unit step in y ($\Delta y = 1$), $\Delta x = 1/m$.

Case 1: $|m| \leq 1$ ($\Delta x \geq \Delta y$) — step in x

$$\begin{aligned}x_{k+1} &= x_k + 1 \\y_{k+1} &= y_k + m\end{aligned}$$

Case 2: $|m| > 1$ ($\Delta y > \Delta x$) — step in y

$$\begin{aligned}y_{k+1} &= y_k + 1 \\x_{k+1} &= x_k + 1/m\end{aligned}$$

At each step, the computed (x,y) — which may be fractional — is rounded to the nearest integer to select the actual pixel to plot. The number of steps taken equals whichever of $|\Delta x|$ or $|\Delta y|$ is larger, ensuring one pixel is plotted for every unit of the dominant axis.

Q4. Compare DDA Algorithm and Bresenham's Line Algorithm. [8 Marks]

Aspect	DDA Algorithm	Bresenham's Algorithm
Calculations	Uses floating-point arithmetic (real numbers) at each step	Uses only integer arithmetic (addition/subtraction)
Speed	Slower — floating point operations and rounding at every step	Faster — simpler integer operations, more efficient
Accuracy	Accumulated rounding errors can cause slight inaccuracy over long lines	More accurate — decision parameter avoids cumulative error
Complexity	Simple to understand and implement	Slightly more complex derivation, but efficient in practice
Hardware Suitability	Less suitable for hardware implementation	Widely used in hardware/graphics libraries due to efficiency

Q5 & Q6. Derive Bresenham's line drawing algorithm for $|m| < 1$ and $|m| > 1$. [8 Marks]

In Simple English

Bresenham's trick is to avoid slow decimal math entirely — it just keeps a running 'error' counter (decision parameter) made of whole numbers, and uses its sign to decide whether the line has drifted closer to the pixel above or the one directly across.

Derivation for $|m| < 1$ (stepping in x)

Let the line go from (x_1, y_1) to (x_2, y_2) . Define $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$. For $|m| < 1$, we step x by 1 each time and decide whether y stays the same or increments by 1.

At step k, suppose the last plotted pixel is (x_k, y_k) . The next x-coordinate is $x_{k+1} = x_k + 1$. The actual line's y-value at this x is:

$$y = y_k + m(x_{k+1} - x_k) = y_k + m$$

We must decide whether to plot (x_{k+1}, y_k) or (x_{k+1}, y_{k+1}) — whichever is closer to the true line value y. Define:

$$\begin{aligned} d1 &= y - y_k && \text{(distance to lower pixel)} \\ d2 &= (y_{k+1}) - y && \text{(distance to upper pixel)} \end{aligned}$$

We choose the upper pixel if $d1 > d2$ (i.e., $d1 - d2 > 0$). Substituting $y = y_k + \Delta y / \Delta x$:

$$\begin{aligned} d1 - d2 &= 2m(x_{k+1}) - 2y_k + 2y_k \dots \text{simplified using } m = \Delta y / \Delta x: \\ d1 - d2 &= 2(\Delta y / \Delta x)(x_{k+1}) - 2y_k - 1 \end{aligned}$$

Multiplying through by Δx (positive, since $|m| < 1$ means $\Delta x > 0$) to eliminate the fraction, we define the decision parameter:

$$p_k = \Delta x(d1 - d2) = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c$$

which simplifies to the well-known recurrence relation:

$$\text{Initial decision parameter: } p_0 = 2\Delta y - \Delta x$$

$$\begin{aligned} \text{If } p_k < 0: & \quad \text{plot } (x_{k+1}, y_k); & \quad p_{k+1} &= p_k + 2\Delta y \\ \text{If } p_k \geq 0: & \quad \text{plot } (x_{k+1}, y_{k+1}); & \quad p_{k+1} &= p_k + 2\Delta y - 2\Delta x \end{aligned}$$

Since Δx , Δy , and 2 are all integers, p_k is always an integer — completely avoiding floating-point arithmetic.

Derivation for $|m| > 1$ (stepping in y)

By symmetry, when $|m| > 1$ the roles of x and y swap: we increment y by 1 at each step and decide whether x stays the same or increments, using an analogous decision parameter:

$$\text{Initial decision parameter: } p_0 = 2\Delta x - \Delta y$$

$$\begin{aligned} \text{If } p_k < 0: & \quad \text{plot } (x_k, y_{k+1}); & \quad p_{k+1} &= p_k + 2\Delta x \\ \text{If } p_k \geq 0: & \quad \text{plot } (x_{k+1}, y_{k+1}); & \quad p_{k+1} &= p_k + 2\Delta x - 2\Delta y \end{aligned}$$

Q7. Digitize the pixel positions of a line from (20,10) to (26,15) using Bresenham's line drawing algorithm. [8 Marks]

Step 1: Compute $\Delta x, \Delta y$

$\Delta x = x_2 - x_1 = 26 - 20 = 6$
 $\Delta y = y_2 - y_1 = 15 - 10 = 5$
 Since $\Delta y < \Delta x$, slope $|m| = 5/6 < 1 \rightarrow$ step in x

Step 2: Initial decision parameter

$p_0 = 2\Delta y - \Delta x = 2(5) - 6 = 4$
 $2\Delta y = 10 \quad 2\Delta y - 2\Delta x = 10 - 12 = -2$

Step 3: Iterate (starting point (20,10) is plotted first)

k	p_k	Decision ($p_k \geq 0$?)	Next Point (x_{k+1}, y_{k+1})	p_{k+1}
0	4	Yes $\rightarrow$ y increments	(21, 11)	$4 + 10 - 12 = 2$
1	2	Yes $\rightarrow$ y increments	(22, 12)	$2 + 10 - 12 = 0$
2	0	Yes $\rightarrow$ y increments	(23, 13)	$0 + 10 - 12 = -2$
3	-2	No $\rightarrow$ y same	(24, 13)	$-2 + 10 = 8$
4	8	Yes $\rightarrow$ y increments	(25, 14)	$8 + 10 - 12 = 6$
5	6	Yes $\rightarrow$ y increments	(26, 15)	— (endpoint reached)

Final Pixel Positions

(20,10), (21,11), (22,12), (23,13), (24,13), (25,14), (26,15)

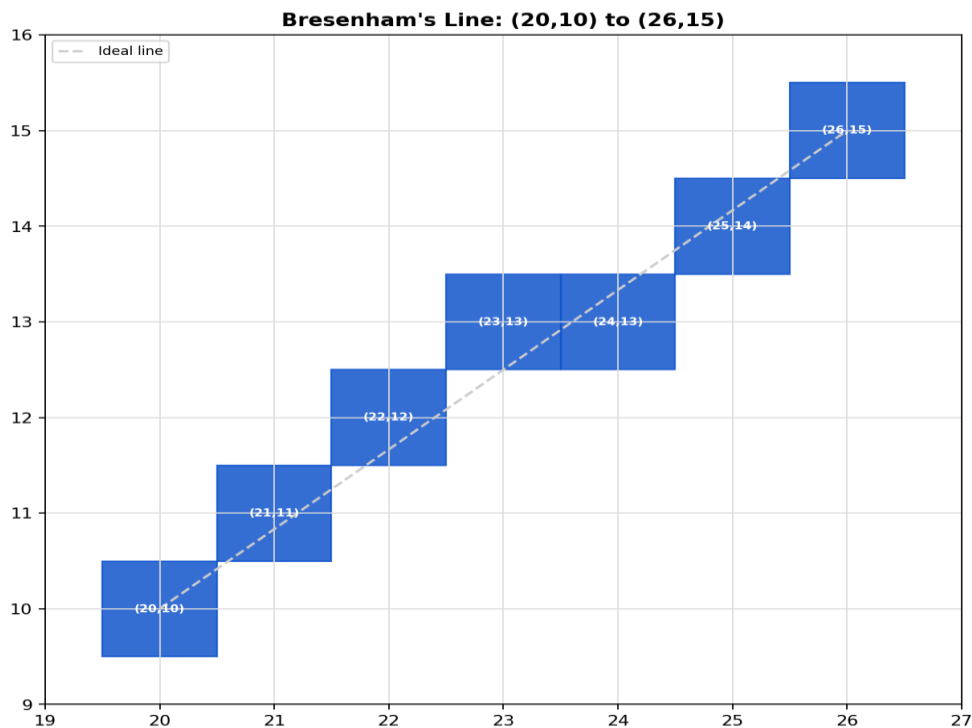


Figure: Bresenham's Line from (20,10) to (26,15)

Q8. Derive the midpoint circle drawing algorithm. [8 Marks]

In Simple English

A circle has 8-way symmetry — if we figure out just one 8th slice (45 degrees) of it, we can mirror those points to draw the whole circle. The midpoint algorithm decides, pixel by pixel, whether the true circle passes closer to one candidate pixel or another, using only integer math.

Derivation

Consider a circle centered at the origin with radius r . Its equation is:

$$f(x,y) = x^2 + y^2 - r^2 = 0$$

We compute only the octant from angle 90° to 45° (where the slope of the curve has magnitude ≤ 1), starting at $(0, r)$. At each step, x always increments by 1; we must decide whether y stays the same or decrements by 1. The decision is based on the midpoint between the two candidate pixels:

$$\text{Midpoint } M = (x_k + 1, y_k - 1/2)$$

Define the decision parameter as the circle function evaluated at this midpoint:

$$p_k = f(x_{k+1}, y_{k-1/2}) = (x_{k+1})^2 + (y_{k-1/2})^2 - r^2$$

- If $p_k < 0$: the midpoint lies inside the circle $\rightarrow$ the pixel above (y_k) is closer to the true circle $\rightarrow$ choose (x_{k+1}, y_k) , y unchanged
- If $p_k \geq 0$: the midpoint lies outside the circle $\rightarrow$ the pixel below (y_{k-1}) is closer $\rightarrow$ choose (x_{k+1}, y_{k-1}) , decrement y

To find the recursive update, evaluate p_{k+1} using the same formula at the next step, and subtract p_k to get an incremental formula (avoiding recomputing the whole expression each time):

$$\begin{aligned} \text{If } p_k < 0 \text{ (y unchanged):} \\ p_{k+1} &= p_k + 2x_{k+1} + 1 \quad (\text{since } x_{k+1} = x_k + 1) \end{aligned}$$

$$\begin{aligned} \text{If } p_k \geq 0 \text{ (y decremented):} \\ p_{k+1} &= p_k + 2x_{k+1} + 1 - 2y_{k+1} \end{aligned}$$

The initial decision parameter is found by evaluating p at the first midpoint $(1, r - 1/2)$:

$$\begin{aligned} p_0 &= f(1, r-1/2) = 1 + (r-1/2)^2 - r^2 = 1 + r^2 - r + 1/4 - r^2 = 5/4 - r \\ \text{Since } p_0 \text{ must be an integer, we round: } p_0 &= 1 - r \end{aligned}$$

The loop continues incrementing x and updating y/p until $x \geq y$ (i.e., the 45° point is reached), after which the remaining seven octants are obtained purely by symmetry — for every computed point (x,y) , the seven symmetric points $(\pm x, \pm y)$ and $(\pm y, \pm x)$ also lie on the circle.

Q9. Digitize one octant of the circle using the midpoint circle algorithm, center (10,20), radius 10. [8 Marks]

Step 1: Initialize

```
x = 0, y = r = 10
p0 = 1 - r = 1 - 10 = -9
```

Step 2: Iterate while $x < y$

k	pk	Decision	(xk+1, yk+1)	pk+1
0	-9	$pk < 0 \rightarrow y$ same	(1, 10)	$-9 + 2(1) + 1 = -6$
1	-6	$pk < 0 \rightarrow y$ same	(2, 10)	$-6 + 2(2) + 1 = -1$
2	-1	$pk < 0 \rightarrow y$ same	(3, 10)	$-1 + 2(3) + 1 = 6$
3	6	$pk \geq 0 \rightarrow y-1$	(4, 9)	$6 + 2(4) + 1 - 2(9) = -3$
4	-3	$pk < 0 \rightarrow y$ same	(5, 9)	$-3 + 2(5) + 1 = 8$
5	8	$pk \geq 0 \rightarrow y-1$	(6, 8)	$8 + 2(6) + 1 - 2(8) = 5$
6	5	$pk \geq 0 \rightarrow y-1$	(7, 7)	stop ($x=y$)

Octant Points (relative to center)

```
(0,10), (1,10), (2,10), (3,10), (4,9), (5,9), (6,8), (7,7)
```

Translating to Actual Center (10, 20)

Add the center coordinates (10, 20) to each relative point:

Relative (x,y)	Actual Pixel (x+10, y+20)
(0,10)	(10,30)
(1,10)	(11,30)
(2,10)	(12,30)
(3,10)	(13,30)
(4,9)	(14,29)
(5,9)	(15,29)
(6,8)	(16,28)
(7,7)	(17,27)

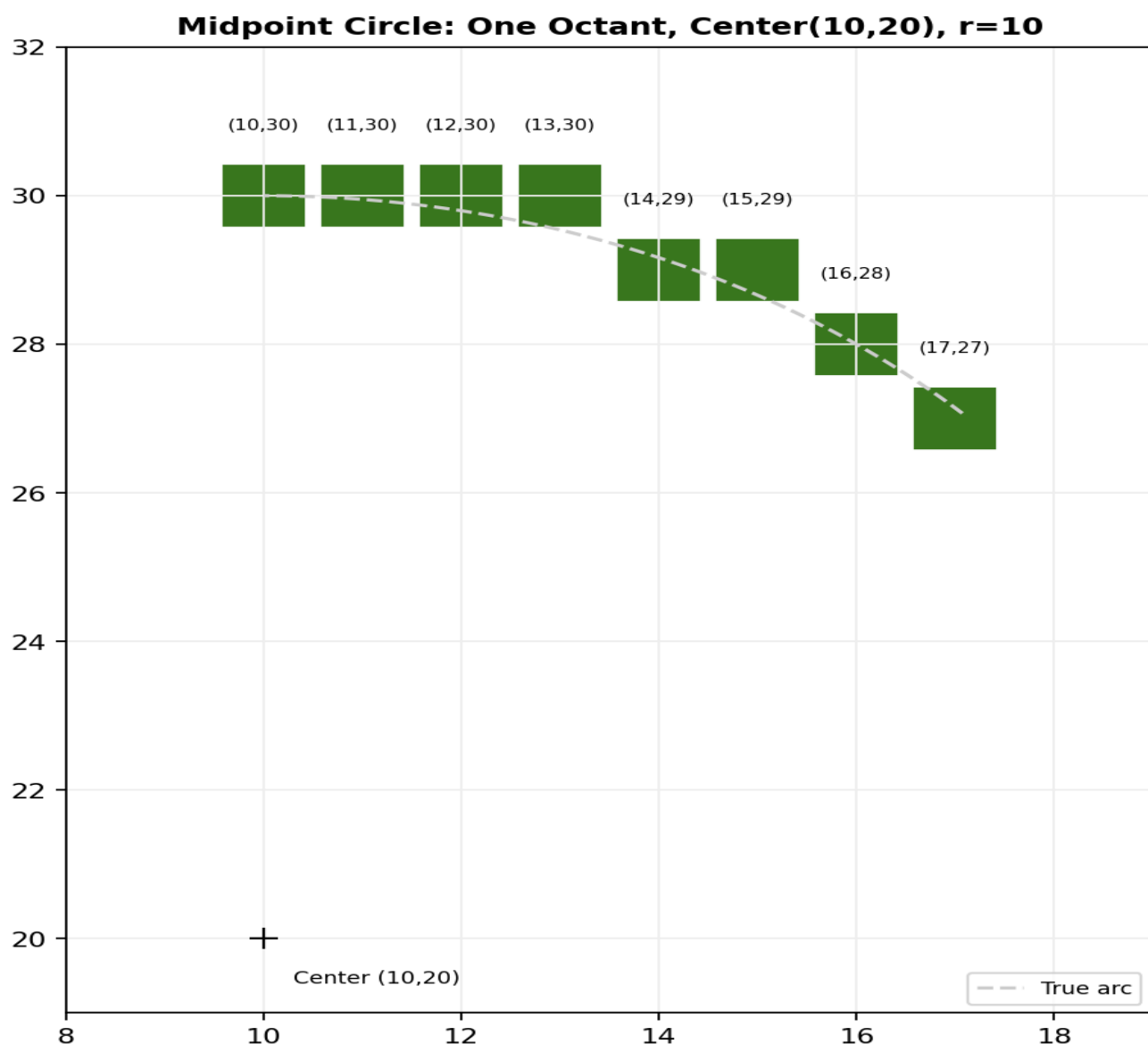


Figure: Midpoint Circle Octant, Center (10,20), r=10

Q10. Digitize the pixel positions of a circle with radius 7 and centre (70,90) in the first and second quadrants using the mid-point circle algorithm. [8 Marks]

Step 1 & 2: Compute one octant for $r = 7$

$$x = 0, y = 7, p_0 = 1 - 7 = -6$$

k	p_k	Decision	(x_{k+1}, y_{k+1})	p_{k+1}
0	-6	$p_k < 0 \rightarrow y \text{ same}$	(1,7)	$-6 + 2(1) + 1 = -3$
1	-3	$p_k < 0 \rightarrow y \text{ same}$	(2,7)	$-3 + 2(2) + 1 = 2$
2	2	$p_k \geq 0 \rightarrow y-1$	(3,6)	$2 + 2(3) + 1 - 2(6) = -3$
3	-3	$p_k < 0 \rightarrow y \text{ same}$	(4,6)	$-3 + 2(4) + 1 = 6$
4	6	$p_k \geq 0 \rightarrow y-1$	(5,5)	stop ($x=y$)

Octant points (0° to 45°): (0,7), (1,7), (2,7), (3,6), (4,6), (5,5)

Step 3: Extend to full first quadrant using symmetry $(x,y) \leftrightarrow (y,x)$

Reflecting each octant point about the 45° line (swap x and y) gives the rest of the first quadrant:

First-quadrant relative points:

(0,7), (1,7), (2,7), (3,6), (4,6), (5,5), (6,4), (6,3), (7,2), (7,1), (7,0)

Step 4: Mirror into second quadrant ($x \rightarrow -x$)

Second-quadrant relative points:

(0,7), (-1,7), (-2,7), (-3,6), (-4,6), (-5,5), (-6,4), (-6,3), (-7,2), (-7,1), (-7,0)

Step 5: Translate by center (70, 90) — add 70 to x, 90 to y

First Quadrant (Actual)	Second Quadrant (Actual)
(70,97)	(70,97)
(71,97)	(69,97)
(72,97)	(68,97)
(73,96)	(67,96)
(74,96)	(66,96)
(75,95)	(65,95)
(76,94)	(64,94)
(76,93)	(64,93)
(77,92)	(63,92)
(77,91)	(63,91)
(77,90)	(63,90)

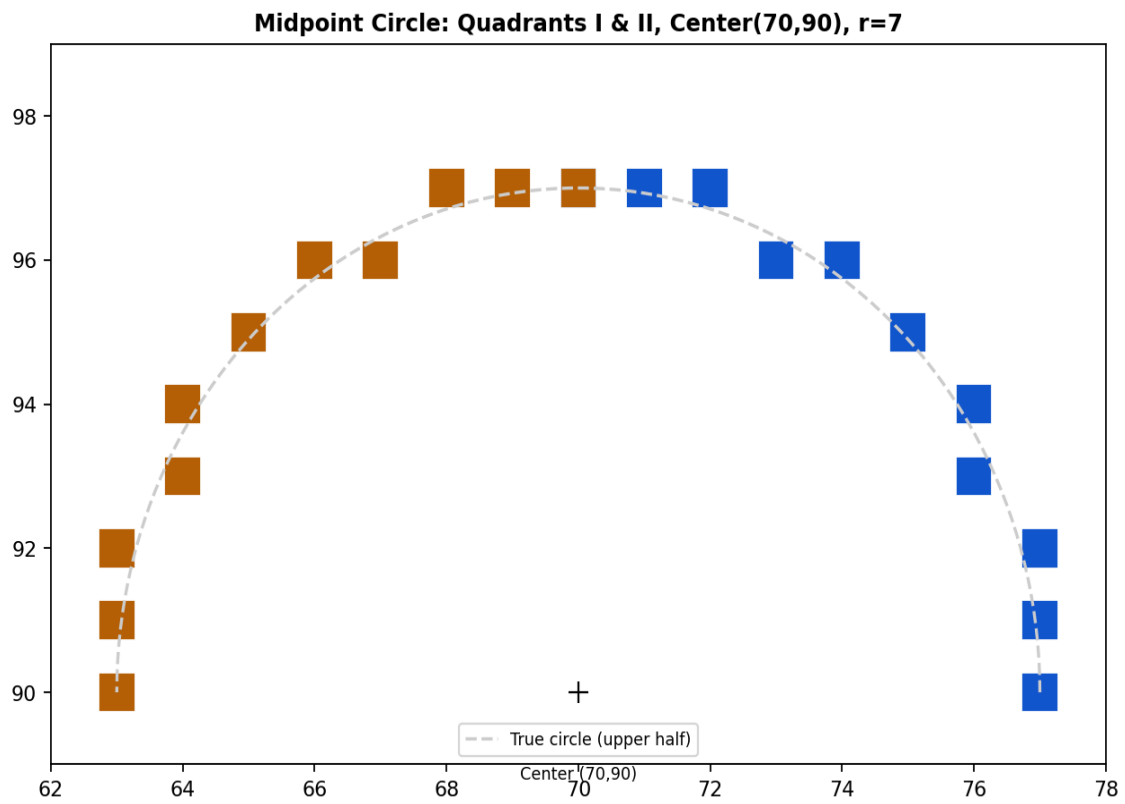


Figure: Circle Center (70,90), $r=7$ — Quadrants I & II

Q11. Draw a circle of radius 7 units centred at (100,100) using the mid-point circle algorithm.

[8 Marks]

The octant calculation for $r = 7$ is identical to Q10 above (the radius is the same): the computed octant points are (0,7), (1,7), (2,7), (3,6), (4,6), (5,5).

Applying 8-Way Symmetry

For each computed point (x, y) in the first octant, the full circle is obtained using all 8 symmetric points:

$(\pm x, \pm y)$ and $(\pm y, \pm x)$

Translating the First-Octant Points to Center (100,100)

Relative (x,y)	Actual Pixel (x+100, y+100)
(0,7)	(100,107)
(1,7)	(101,107)
(2,7)	(102,107)
(3,6)	(103,106)
(4,6)	(104,106)
(5,5)	(105,105)

The remaining 7 octants of the actual circle are obtained the same way — reflecting each relative (x,y) pair through the symmetry formulas above, then adding the center offset (100,100) to every reflected point, giving the complete 44-point digitized circle (since circumference $\approx 2\pi r \approx 44$ for $r=7$).

Q12. How can the decision parameter be used to draw a circle? Calculate all pixels of a circle in the first quadrant, radius=5, center (10,5). [8 Marks]

How the Decision Parameter Works

The decision parameter p_k is a purely-integer quantity derived from the circle equation evaluated at the midpoint between two candidate pixels. Its sign tells us which candidate pixel is closer to the true circular arc: a negative p_k means the midpoint lies inside the circle (so the 'straight-across' pixel is closer), while $p_k \geq 0$ means the midpoint lies outside (so the 'diagonally-down' pixel is closer). This lets the algorithm choose pixels using only additions and sign checks — no square roots or floating-point trigonometry.

Step 1 & 2: Initialize for $r = 5$

$$x = 0, y = 5, p_0 = 1 - r = 1 - 5 = -4$$

k	p_k	Decision	(x_{k+1}, y_{k+1})	p_{k+1}
0	-4	$p_k < 0 \rightarrow y$ same	(1,5)	$-4 + 2(1) + 1 = -1$
1	-1	$p_k < 0 \rightarrow y$ same	(2,5)	$-1 + 2(2) + 1 = 4$
2	4	$p_k \geq 0 \rightarrow y-1$	(3,4)	$4 + 2(3) + 1 - 2(4) = 3$
3	3	$p_k \geq 0 \rightarrow y-1$	(4,3)	stop ($x=y$)

Octant points: (0,5), (1,5), (2,5), (3,4), (4,3)

Extending to the Full First Quadrant (symmetry $x \leftrightarrow y$)

First-quadrant relative points: (0,5), (1,5), (2,5), (3,4), (4,3), (5,2), (5,1), (5,0)

Translating to Center (10,5)

Relative (x,y)	Actual Pixel (x+10, y+5)
(0,5)	(10,10)
(1,5)	(11,10)
(2,5)	(12,10)
(3,4)	(13,9)
(4,3)	(14,8)
(5,2)	(15,7)
(5,1)	(15,6)
(5,0)	(15,5)

Q13. Define Scan Conversion. [8 Marks]

In Simple English

Scan conversion is the process of turning a mathematical shape (a line, circle, or polygon) into the actual pixels that must be lit up on a screen.

Technical Explanation

Scan conversion (also called rasterization) is the process of representing continuous, mathematically-defined graphics objects (lines, circles, curves, filled areas) as a discrete collection of pixel intensity values stored in the frame buffer, so that the object can be displayed on a raster (pixel-based) display device. Algorithms such as DDA, Bresenham's line algorithm, and the midpoint circle algorithm are all scan-conversion algorithms.

4. 2D & 3D Geometric Transformations

Q1 & Q2. What is homogeneous coordinate system? Why is it important for composite transformation? [8 Marks]

In Simple English

Normally, moving (translating) a shape can't be done with simple matrix multiplication like scaling or rotating can. Homogeneous coordinates add one extra 'fake' number to each point so that even moving a shape becomes a matrix multiplication — which means we can chain (combine) any sequence of transformations into a single matrix.

Technical Explanation

In homogeneous coordinates, a 2D point (x, y) is represented as a 3-element vector (x, y, 1), and a 3D point (x, y, z) is represented as (x, y, z, 1). This extra coordinate allows translation — which is normally an additive operation — to be expressed as a matrix multiplication, exactly like scaling, rotation, and shearing.

Importance for Composite Transformation

Because every transformation (translation, scaling, rotation, shearing) can now be written as a matrix multiplication in the same homogeneous form, multiple transformations can be combined (composed) into a single resultant matrix simply by multiplying the individual matrices together:

$$P' = [T_n] \times [T_{n-1}] \times \dots \times [T_2] \times [T_1] \times P$$

This single composite matrix can then be applied to every vertex of an object just once, which is far more efficient than applying each transformation separately in sequence, especially for objects with many vertices.

Q3. Explain the basic 2D geometric transformations with matrix representation. [8 Marks]

In Simple English

There are a few basic ways to change a shape's position, size, or angle: sliding it (translation), stretching it (scaling), spinning it (rotation), and slanting it (shearing).

1. Translation

Moves every point by a fixed displacement (tx, ty):

$$\begin{array}{l} x' = x + tx \\ y' = y + ty \end{array} \rightarrow \begin{array}{|c|c|c|} \hline 1 & 0 & tx \\ \hline 0 & 1 & ty \\ \hline 0 & 0 & 1 \\ \hline \end{array} \quad (\text{homogeneous matrix})$$

2. Scaling

Changes the size of an object by factors sx, sy about the origin:

$$\begin{array}{l} x' = x \cdot sx \\ y' = y \cdot sy \end{array} \rightarrow \begin{array}{|c|c|c|} \hline sx & 0 & 0 \\ \hline 0 & sy & 0 \\ \hline 0 & 0 & 1 \\ \hline \end{array}$$

3. Rotation (about origin, angle θ , anti-clockwise)

$$\begin{array}{l} x' = x \cdot \cos\theta - y \cdot \sin\theta \\ y' = x \cdot \sin\theta + y \cdot \cos\theta \end{array} \rightarrow \begin{array}{|c|c|c|} \hline \cos\theta & -\sin\theta & 0 \\ \hline \sin\theta & \cos\theta & 0 \\ \hline 0 & 0 & 1 \\ \hline \end{array}$$

4. Shearing

Slants the shape along the x or y axis:

$$\begin{array}{l} \text{X-Shear: } x' = x + shx \cdot y, \quad y' = y \end{array} \quad \begin{array}{|c|c|c|} \hline 1 & shx & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 1 \\ \hline \end{array}$$

Q4. Derive the composite transformation matrix for two successive translations. [8 Marks]

In Simple English

If you slide a shape once, and then slide it again, the total effect is exactly the same as just adding up both slide distances and doing it in one move.

Derivation

Let the first translation be by $(tx1, ty1)$ and the second by $(tx2, ty2)$. In homogeneous matrix form:

$$T1 = \begin{bmatrix} 1 & 0 & tx1 \\ 0 & 1 & ty1 \\ 0 & 0 & 1 \end{bmatrix} \quad T2 = \begin{bmatrix} 1 & 0 & tx2 \\ 0 & 1 & ty2 \\ 0 & 0 & 1 \end{bmatrix}$$

Applying T1 first, then T2, the composite transformation matrix is the product $T2 \cdot T1$:

$$T2 \cdot T1 = \begin{bmatrix} 1 & 0 & tx2 \\ 0 & 1 & ty2 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & tx1 \\ 0 & 1 & ty1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & (tx1+tx2) \\ 0 & 1 & (ty1+ty2) \\ 0 & 0 & 1 \end{bmatrix}$$

The result is itself a single translation matrix with displacement $(tx1+tx2, ty1+ty2)$. This proves that two successive translations are additive — combining into one translation whose components are simply the sum of the individual displacements.

Q5 & Q6. Prove that two successive rotations are additive, and that two successive translations are additive. [8 Marks]

Proof: Two Successive Translations Are Additive

(Derived fully above in Q4.) Multiplying two translation matrices $T(tx1,ty1)$ and $T(tx2,ty2)$ yields $T(tx1+tx2, ty1+ty2)$ — a single translation whose offsets are the sum of the two individual offsets. Hence translations combine additively.

Proof: Two Successive Rotations Are Additive

Let $R(\theta1)$ and $R(\theta2)$ be two rotation matrices about the origin:

$$R(\theta1) = \begin{vmatrix} \cos\theta1 & -\sin\theta1 & 0 \\ \sin\theta1 & \cos\theta1 & 0 \\ 0 & 0 & 1 \end{vmatrix} \quad R(\theta2) = \begin{vmatrix} \cos\theta2 & -\sin\theta2 & 0 \\ \sin\theta2 & \cos\theta2 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Multiplying $R(\theta2) \cdot R(\theta1)$, and applying the trigonometric angle-sum identities $\cos(A+B)=\cos A \cdot \cos B - \sin A \cdot \sin B$ and $\sin(A+B)=\sin A \cdot \cos B + \cos A \cdot \sin B$ to each resulting element:

$$\begin{aligned} R(\theta2) \cdot R(\theta1) &= \begin{vmatrix} \cos\theta2\cos\theta1 - \sin\theta2\sin\theta1 & -(\cos\theta2\sin\theta1 + \sin\theta2\cos\theta1) & 0 \\ \sin\theta2\cos\theta1 + \cos\theta2\sin\theta1 & \cos\theta2\cos\theta1 - \sin\theta2\sin\theta1 & 0 \\ 0 & 0 & 1 \end{vmatrix} \\ &= \begin{vmatrix} \cos(\theta1+\theta2) & -\sin(\theta1+\theta2) & 0 \\ \sin(\theta1+\theta2) & \cos(\theta1+\theta2) & 0 \\ 0 & 0 & 1 \end{vmatrix} = R(\theta1+\theta2) \end{aligned}$$

The product simplifies exactly to a single rotation matrix $R(\theta1+\theta2)$. This proves that two successive rotations about the same point are additive — equivalent to one rotation by the sum of the two angles.

Q7. Find the composite transformation matrix for anti-clockwise rotation of 45° about point (2,3). Use it to rotate triangle ABC with vertices A(0,0), B(6,2), C(3,4). [8 Marks]

Step 1: General Formula for Rotation about an Arbitrary Pivot (xp, yp)

Rotation about a pivot point (other than the origin) is built as a composite of three transformations: translate the pivot to the origin, rotate about the origin, then translate back:

$$[T] = T(x_p, y_p) \cdot R(\theta) \cdot T(-x_p, -y_p)$$

Expanding this product gives the general point-transformation formulas:

$$\begin{aligned}x' &= x_p + (x - x_p)\cos\theta - (y - y_p)\sin\theta \\y' &= y_p + (x - x_p)\sin\theta + (y - y_p)\cos\theta\end{aligned}$$

Step 2: Substitute $\theta = 45^\circ$, $(x_p, y_p) = (2, 3)$

$$\cos 45^\circ = \sin 45^\circ = 0.7071$$

$$\begin{array}{ccc|ccc} \text{Composite Matrix } [T] = & & & & & & \\ \begin{vmatrix} \cos\theta & -\sin\theta & x_p(1-\cos\theta)+y_p\sin\theta \\ \sin\theta & \cos\theta & y_p(1-\cos\theta)-x_p\sin\theta \\ 0 & 0 & 1 \end{vmatrix} & = & \begin{vmatrix} 0.7071 & -0.7071 & 2.7071 \\ 0.7071 & 0.7071 & -0.5355 \\ 0 & 0 & 1 \end{vmatrix} \end{array}$$

Step 3: Apply to Each Vertex

$$\begin{aligned}A(0,0): \quad & x - x_p = -2, \quad y - y_p = -3 \\x' &= 2 + (-2)(0.7071) - (-3)(0.7071) = 2 + 0.7071(1) = 2.7071 \\y' &= 3 + (-2)(0.7071) + (-3)(0.7071) = 3 + 0.7071(-5) = -0.5355 \\&\rightarrow A' = (2.71, -0.54)\end{aligned}$$

$$\begin{aligned}B(6,2): \quad & x - x_p = 4, \quad y - y_p = -1 \\x' &= 2 + 4(0.7071) - (-1)(0.7071) = 2 + 0.7071(5) = 5.5355 \\y' &= 3 + 4(0.7071) + (-1)(0.7071) = 3 + 0.7071(3) = 5.1213 \\&\rightarrow B' = (5.54, 5.12)\end{aligned}$$

$$\begin{aligned}C(3,4): \quad & x - x_p = 1, \quad y - y_p = 1 \\x' &= 2 + 1(0.7071) - 1(0.7071) = 2 \\y' &= 3 + 1(0.7071) + 1(0.7071) = 4.4142 \\&\rightarrow C' = (2, 4.41)\end{aligned}$$

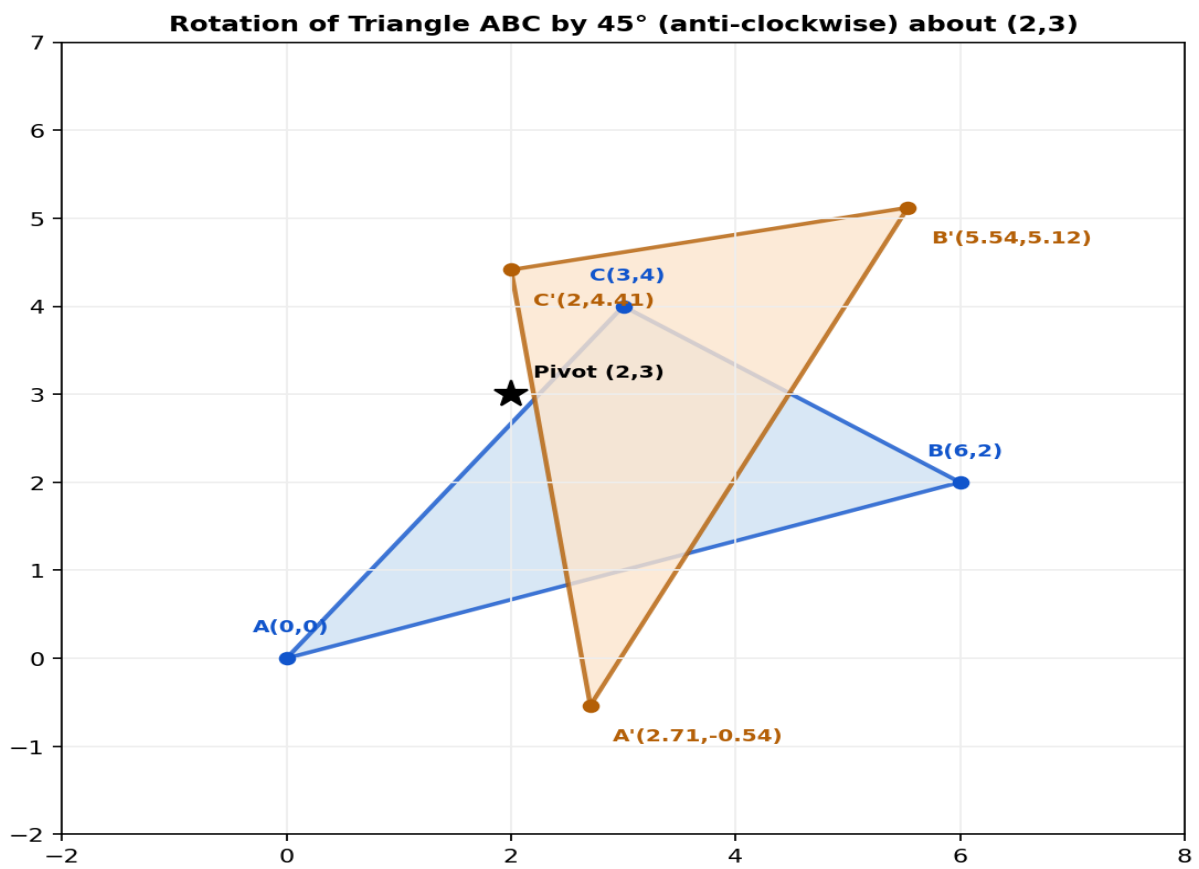


Figure: Triangle ABC rotated 45° anti-clockwise about pivot (2,3)

Q8. Find the final coordinates of polygon A(3,4), B(5,4), C(5,2), D(3,2) after rotation by 45° about fixed point (2,3). [8 Marks]

(Note: the coordinate for D was given as (3,4) in the source, which duplicates A — this is treated as a typographical error, and D=(3,2) is used to correctly complete the rectangle ABCD.)

Using the same rotation formula as Q7 ($\theta=45^\circ$, pivot (2,3), $\cos\theta=\sin\theta=0.7071$):

A(3,4): $x-x_p=1$, $y-y_p=1$

$$x' = 2 + 0.7071(1) - 0.7071(1) = 2$$

$$y' = 3 + 0.7071(1) + 0.7071(1) = 4.4142 \rightarrow A' = (2, 4.41)$$

B(5,4): $x-x_p=3$, $y-y_p=1$

$$x' = 2 + 0.7071(3) - 0.7071(1) = 2+1.4142 = 3.4142$$

$$y' = 3 + 0.7071(3) + 0.7071(1) = 3+2.8284 = 5.8284 \rightarrow B' = (3.41, 5.83)$$

C(5,2): $x-x_p=3$, $y-y_p=-1$

$$x' = 2 + 0.7071(3) - 0.7071(-1) = 2+2.8284 = 4.8284$$

$$y' = 3 + 0.7071(3) + 0.7071(-1) = 3+1.4142 = 4.4142 \rightarrow C' = (4.83, 4.41)$$

D(3,2): $x-x_p=1$, $y-y_p=-1$

$$x' = 2 + 0.7071(1) - 0.7071(-1) = 2+1.4142 = 3.4142$$

$$y' = 3 + 0.7071(1) + 0.7071(-1) = 3+0 = 3 \rightarrow D' = (3.41, 3)$$

Q9. Triangle ABC with A(0,5), B(9,12), C(3,14): (i) Rotate about origin (ii) Rotate about fixed point B(5,0) by 90°. [8 Marks]

(Note: the source question omits the rotation angle for part (i) and gives the fixed point for part (ii) as B(5,0), inconsistent with the triangle's actual vertex B(9,12) — likely two exam sub-parts merged during transcription. Both parts are solved below using the standard rotation-formula method, assuming a 90° anti-clockwise rotation for part (i) as is conventional for such paired questions.)

(i) Rotate about the Origin by 90°

For rotation about the origin by $\theta=90^\circ$ ($\cos\theta=0$, $\sin\theta=1$): $x' = -y$, $y' = x$

$$\begin{aligned} A(0,5) &\rightarrow A' = (-5, 0) \\ B(9,12) &\rightarrow B' = (-12, 9) \\ C(3,14) &\rightarrow C' = (-14, 3) \end{aligned}$$

(ii) Rotate about Fixed Point (5,0) by 90°

Using $x' = x_p - (y - y_p)$, $y' = y_p + (x - x_p)$ for $\theta=90^\circ$, with $(x_p, y_p)=(5,0)$:

$$\begin{aligned} A(0,5): \quad x' &= 5 - (5 - 0) = 0, & y' &= 0 + (0 - 5) = -5 & \rightarrow A' &= (0, -5) \\ B(9,12): \quad x' &= 5 - (12 - 0) = -7, & y' &= 0 + (9 - 5) = 4 & \rightarrow B' &= (-7, 4) \\ C(3,14): \quad x' &= 5 - (14 - 0) = -9, & y' &= 0 + (3 - 5) = -2 & \rightarrow C' &= (-9, -2) \end{aligned}$$

Q10. Right-angle triangle A(1,0), B(5,0), C(5,5): (i) Scale ABC to double its size at the origin. (ii) Rotate it about fixed point B(5,0) by 90°. [8 Marks]

(i) Scaling by Factor 2 about the Origin

Using $x'=2x$, $y'=2y$:

$$\begin{aligned} A(1,0) &\rightarrow A' = (2, 0) \\ B(5,0) &\rightarrow B' = (10, 0) \\ C(5,5) &\rightarrow C' = (10, 10) \end{aligned}$$

(ii) Rotate the Original Triangle about Fixed Point B(5,0) by 90°

Using $x' = x_p - (y - y_p)$, $y' = y_p + (x - x_p)$ for $\theta=90^\circ$, with $(x_p, y_p)=(5,0)$:

$$\begin{aligned} A(1,0): \quad x' &= 5 - (0 - 0) = 5, \quad y' = 0 + (1 - 5) = -4 \quad \rightarrow A' = (5, -4) \\ B(5,0): \quad x' &= 5 - (0 - 0) = 5, \quad y' = 0 + (5 - 5) = 0 \quad \rightarrow B' = (5, 0) \quad [\text{fixed point} - \text{unchanged, as expected}] \\ C(5,5): \quad x' &= 5 - (5 - 0) = 0, \quad y' = 0 + (5 - 5) = 0 \quad \rightarrow C' = (0, 0) \end{aligned}$$

As a sanity check, the pivot point B(5,0) correctly maps to itself, confirming the rotation formula was applied correctly.

Q11. Rectangle ABCD, center (4,2), reduced to half its size (center unchanged). A(0,0), B(0,4), C(8,4), D(8,0). Find new coordinates and derive the transformation matrix. [8 Marks]

Step 1: General Formula for Scaling about a Fixed Point

To scale about an arbitrary fixed point (xf, yf) rather than the origin, we translate the fixed point to the origin, scale, then translate back:

$$\begin{aligned}x' &= x_f + (x - x_f) \cdot s_x \\y' &= y_f + (y - y_f) \cdot s_y\end{aligned}$$

Step 2: Apply with (xf,yf) = (4,2), sx = sy = 0.5

$$\begin{aligned}A(0,0): \quad x' &= 4 + (0-4)(0.5) = 2, & y' &= 2 + (0-2)(0.5) = 1 & \rightarrow A' &= (2, 1) \\B(0,4): \quad x' &= 4 + (0-4)(0.5) = 2, & y' &= 2 + (4-2)(0.5) = 3 & \rightarrow B' &= (2, 3) \\C(8,4): \quad x' &= 4 + (8-4)(0.5) = 6, & y' &= 2 + (4-2)(0.5) = 3 & \rightarrow C' &= (6, 3) \\D(8,0): \quad x' &= 4 + (8-4)(0.5) = 6, & y' &= 2 + (0-2)(0.5) = 1 & \rightarrow D' &= (6, 1)\end{aligned}$$

Step 3: Composite Transformation Matrix

The combined matrix $T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f)$ works out to:

$$[T] = \begin{vmatrix} s_x & 0 & x_f(1-s_x) \\ 0 & s_y & y_f(1-s_y) \\ 0 & 0 & 1 \end{vmatrix} = \begin{vmatrix} 0.5 & 0 & 2 \\ 0 & 0.5 & 1 \\ 0 & 0 & 1 \end{vmatrix}$$

Result

The new shape A'(2,1), B'(2,3), C'(6,3), D'(6,1) has width 4 and height 2 — exactly half the original 8×4 rectangle's width and height, correctly centered at the same point (4,2). Note: since the original rectangle (8×4) is not a square, uniformly scaling both dimensions by the same factor (0.5) preserves its rectangular shape (4×2) rather than turning it into a square; achieving an actual square would require different scale factors for x and y.

5. Clipping & Viewing Pipeline

Q1. Why do we need clipping? [8 Marks]

In Simple English

A screen or window can only show a limited area. Clipping is the process of cutting away all the parts of a picture that fall outside that visible area, so the computer doesn't waste time trying to draw things you'll never actually see.

Technical Explanation

Clipping is the procedure that identifies and removes (or trims) those portions of a picture that lie outside a specified region — the clipping window — so that only the visible portion is processed and displayed.

Why It Is Needed

- **Efficiency:** Prevents wasted processing time and memory drawing objects (or parts of objects) that fall completely outside the visible display area.
- **Correctness:** Without clipping, primitives could be drawn beyond the screen/window boundary, causing errors, wraparound, or crashes in the rasterization hardware/software.
- **Multiple Viewports:** Applications that show multiple views/windows of a scene need each view's contents clipped to its own boundary.
- **Zooming & Panning:** Any interactive zoom/pan operation requires re-clipping the world against a new window each time the view changes.
- **Hidden-Surface/Ray Tracing:** Clipping against the view volume is also required in 3D rendering to remove objects outside the camera's field of view (view frustum culling).

Q2 & Q3. Explain the Cohen-Sutherland Line Clipping algorithm with a suitable example. [8 Marks]

In Simple English

This algorithm gives every point a 4-digit 'address code' that says whether it's above, below, left, or right of the visible window. By comparing the codes of a line's two endpoints, we can often tell instantly whether the whole line is visible, completely invisible, or needs to be trimmed — and if trimming is needed, we cut it piece by piece against each edge of the window.

Technical Explanation

The Cohen-Sutherland algorithm divides the 2D plane into 9 regions using the four edges of the clipping window (extended infinitely), and assigns each region a unique 4-bit region code (outcode), with bits ordered Top-Bottom-Right-Left:

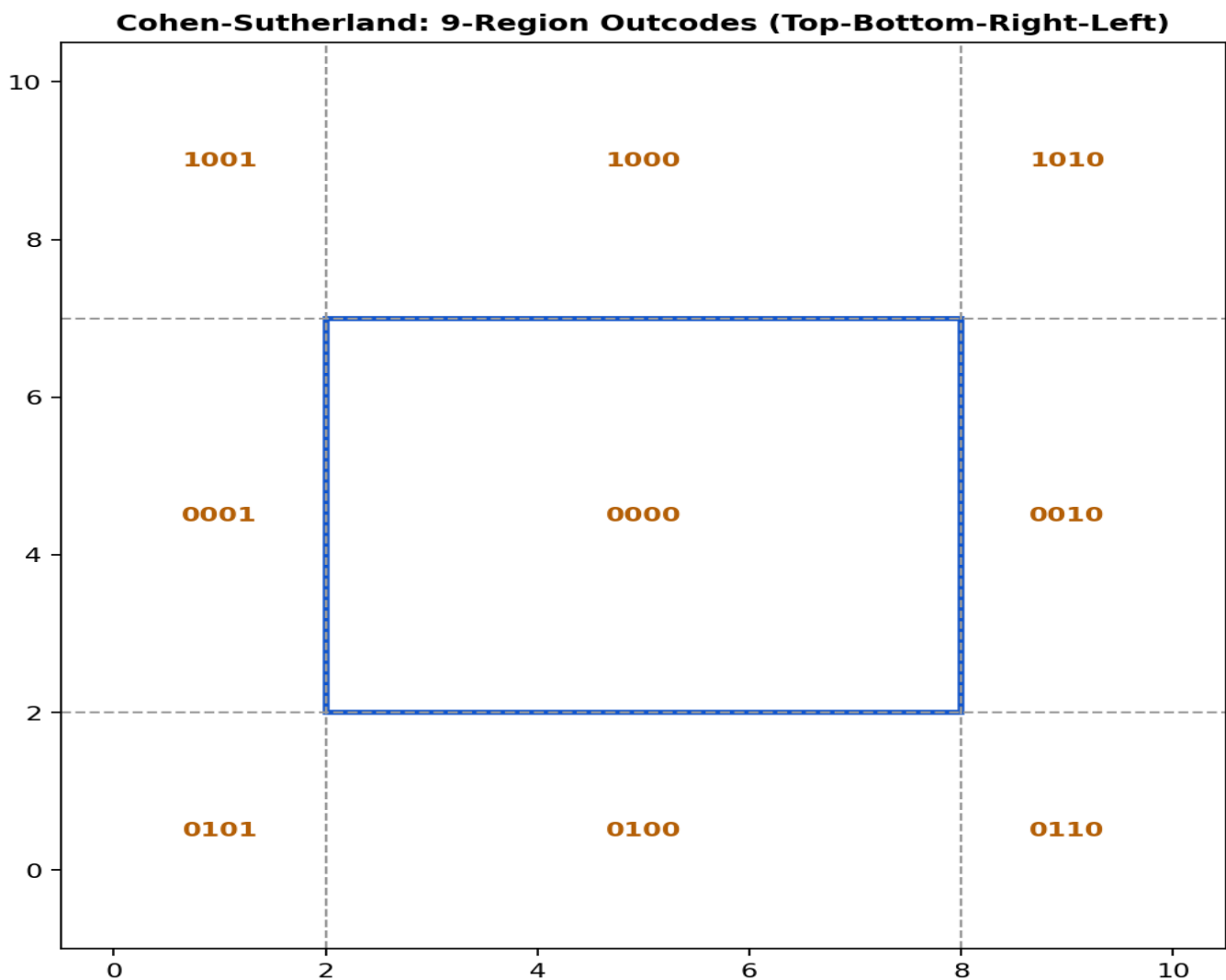


Figure: Cohen-Sutherland 9-Region Outcodes

Each bit is set to 1 based on the point's position:

- Bit 1 (Top): set if $y > y_{\max}$
- Bit 2 (Bottom): set if $y < y_{\min}$
- Bit 3 (Right): set if $x > x_{\max}$
- Bit 4 (Left): set if $x < x_{\min}$

Algorithm Steps

1. Compute the 4-bit region code for both endpoints of the line.
2. Trivial Accept: If both codes are 0000 (both endpoints inside the window), the entire line is inside — accept and draw it as-is.
3. Trivial Reject: If the bitwise AND of the two codes is non-zero, both points share an outside region on the same side — the line is completely outside — reject it.
4. Otherwise (neither trivial accept nor reject): pick an endpoint with a non-zero code, find which window edge it lies outside of, compute the intersection point of the line with that edge (using the line equation), and replace that endpoint with the intersection point.
5. Recompute the region code for the new endpoint and repeat steps 2-4 until the line is either trivially accepted or trivially rejected.

Worked Example

Consider a clipping window with $x_{\min}=2$, $x_{\max}=8$, $y_{\min}=2$, $y_{\max}=7$, and a line from $P1(1,5)$ to $P2(6,10)$.

Region code of $P1(1,5)$: $x < x_{\min} \rightarrow$ Left bit set $\rightarrow$ code = 0001

Region code of $P2(6,10)$: $y > y_{\max} \rightarrow$ Top bit set $\rightarrow$ code = 1000

Bitwise AND(0001, 1000) = 0000 $\rightarrow$ not trivially rejected

Neither code is 0000 $\rightarrow$ not trivially accepted $\rightarrow$ must clip

Since $P1$ is outside on the Left ($x < x_{\min}=2$), we intersect the line with $x=2$:

Slope $m = (10-5)/(6-1) = 1$

At $x=2$: $y = 5 + m(2-1) = 5 + 1 = 6$

New $P1' = (2, 6)$; region code recomputed = 0000 (inside)

Since $P2$ is outside on the Top ($y > y_{\max}=7$), we intersect the updated line with $y=7$:

At $y=7$: $x = 2 + (7-6)/1 = 3$

New $P2' = (3, 7)$; region code recomputed = 0000 (inside)

Both endpoints now have code 0000, so the clipped line segment from $(2,6)$ to $(3,7)$ is accepted and drawn — the portion of the original line lying inside the window.

Q4 & Q5. Explain window-to-viewport transformation. Derive the window-to-viewport transformation matrix. [8 Marks]

In Simple English

The 'window' is the rectangular area of your drawing (in world coordinates) that you want to see. The 'viewport' is the rectangular area on the actual screen where that drawing will appear. This transformation stretches/shrinks and repositions the window's contents to exactly fit inside the viewport.

Window-to-Viewport Transformation

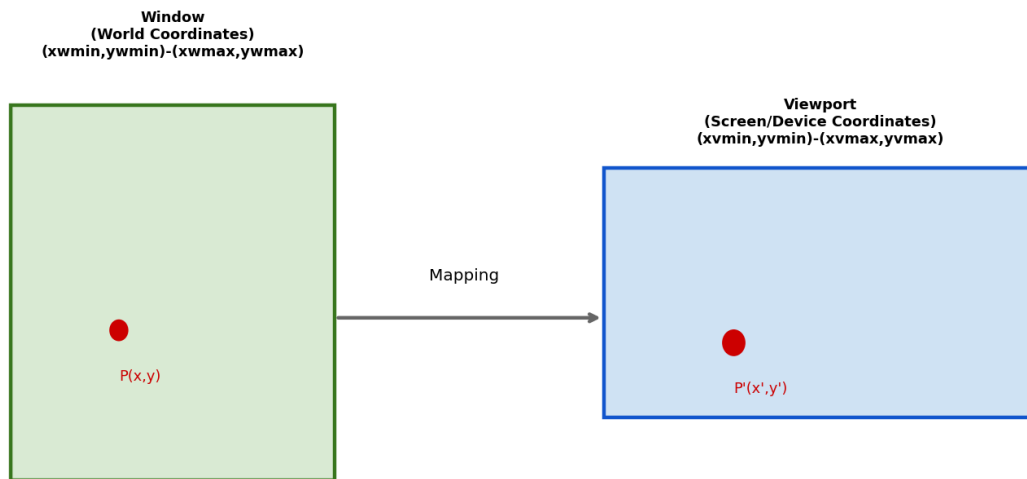


Figure: Window-to-Viewport Transformation

Derivation

Let the window have boundaries (xwmin, ywmin) to (xwmax, ymax), and the viewport have boundaries (xvmin, yvmin) to (xvmax, ymax). We require that a point's relative position within the window is preserved when mapped into the viewport — i.e., the ratio of a point's offset from the window's edge to the window's total size must equal its offset ratio within the viewport:

$$\begin{aligned}\frac{(x - xwmin)}{(xwmax - xwmin)} &= \frac{(x' - xvmin)}{(xvmax - xvmin)} \\ \frac{(y - ywmin)}{(ymax - ywmin)} &= \frac{(y' - yvmin)}{(ymax - yvmin)}\end{aligned}$$

Solving each equation for x' and y' gives the transformation formulas:

$$\begin{aligned}x' &= xvmin + (x - xwmin) \cdot sx & \text{where } sx &= (xvmax - xvmin) / (xwmax - xwmin) \\ y' &= yvmin + (y - ywmin) \cdot sy & \text{where } sy &= (ymax - yvmin) / (ymax - ywmin)\end{aligned}$$

This can be expressed as a composite of three basic transformations: (1) translate the window's corner to the origin, (2) scale by (sx, sy), (3) translate to the viewport's corner:

$$[T] = T(xvmin, yvmin) \cdot S(sx, sy) \cdot T(-xwmin, -ywmin)$$

If $sx = sy$, the relative proportions (aspect ratio) of objects are preserved; if not, objects will appear stretched or squashed.

Q6. Explain the 2D viewing pipeline with its block diagram. [8 Marks]

In Simple English

Getting a picture from the abstract world coordinates onto your actual screen happens in a series of ordered steps, like an assembly line — this sequence is called the viewing pipeline.

Technical Explanation — Stages of the 2D Viewing Pipeline

6. Construct the World-Coordinate Scene: The picture/scene is defined using a convenient world-coordinate system, independent of any specific device.
7. Define the Window: A rectangular window is specified in world coordinates, selecting the portion of the scene to be displayed.
8. Clipping: Objects (or parts of objects) that lie outside the window are clipped away, using algorithms such as Cohen-Sutherland.
9. Window-to-Viewport Transformation: The clipped scene (still in world coordinates) is mapped into the viewport using the scaling and translation derived above.
10. Map to Device Coordinates: Finally, the viewport (normalized device coordinates) is mapped onto the actual physical output device's coordinate system (e.g., a specific monitor's pixel grid) for display.

2D Viewing Pipeline

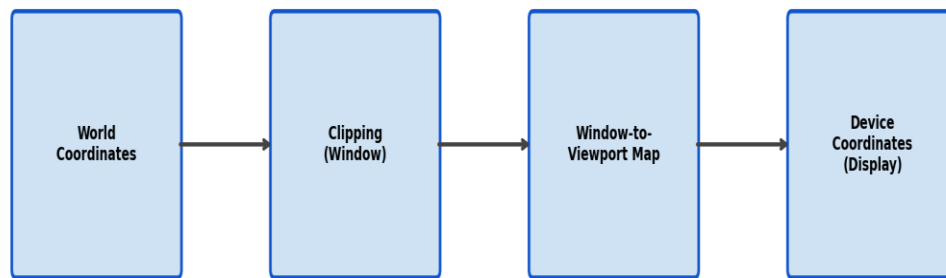


Figure: 2D Viewing Pipeline

6. Projection, Rendering & Shading Models

Q1 & Q4. Define projection. Define parallel and perspective projection. [8 Marks]

In Simple English

Projection is how we flatten a 3D object onto a 2D screen — like the shadow a solid object casts onto a wall.

Technical Explanation

Projection is the process of transforming 3D object coordinates onto a 2D projection (view) plane, by tracing projection lines (projectors) from each point of the object, through/toward a reference point or direction, until they intersect the plane.

- **Parallel Projection:** All projectors are parallel to each other (the center of projection is at infinity). It preserves relative proportions of objects, and is used mainly for engineering/architectural drawings. Sub-types include orthographic and oblique projection.
- **Perspective Projection:** All projectors converge to a single point, the center of projection (COP). It produces realistic images with foreshortening (distant objects appear smaller), similar to how the human eye or a camera perceives the world.

Q2 & Q3. Derive the expression and matrix representation for Perspective Projection. [8 Marks]

In Simple English

Imagine light rays passing from a 3D point, through a single focal point (like a camera's pinhole) and hitting a flat glass plane in front of it. Where each ray crosses the glass is where that point appears in the 2D image — and points farther away end up appearing smaller.

Derivation

Consider the center of projection at the origin $(0,0,0)$, and the view plane placed perpendicular to the z -axis at $z = d$. Let $P(x, y, z)$ be a point in 3D space, and let its projection on the view plane be $P'(x', y', d)$.

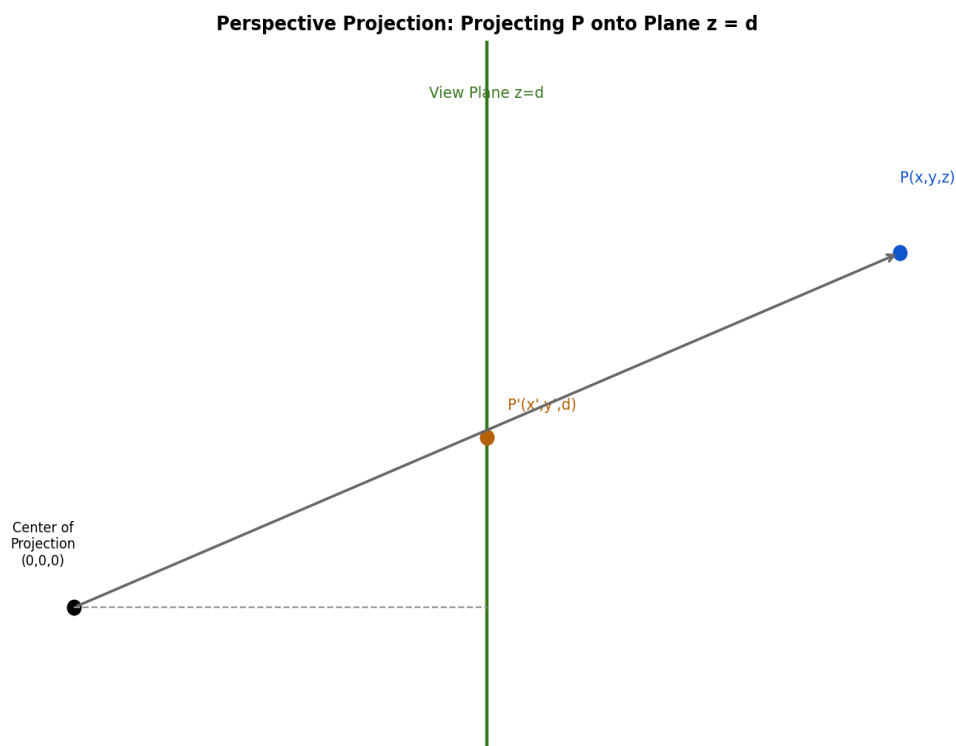


Figure: Perspective Projection onto Plane $z = d$

Since the center of projection, P , and P' are collinear (the projector is a straight line from the origin through P to the view plane), we use similar triangles formed in the x - z plane and y - z plane:

$$\begin{aligned} x'/d &= x/z & \rightarrow & \quad x' = (x \cdot d)/z \\ y'/d &= y/z & \rightarrow & \quad y' = (y \cdot d)/z \end{aligned}$$

Since the plane is at a fixed $z=d$, $z' = d$ for every projected point.

Matrix Representation

Using homogeneous coordinates, this perspective transformation can be represented as a 4×4 matrix. Applying it to the homogeneous point (x, y, z, 1):

$$[P_{\text{per}}] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$$

$$[x \ y \ z \ 1] \cdot [P_{\text{per}}] = [x \ y \ z \ z/d]$$

Converting back from homogeneous form by dividing every component by the last (w) coordinate (z/d) gives the actual perspective-projected point:

$$\begin{aligned} x' &= x / (z/d) = xd/z \\ y' &= y / (z/d) = yd/z \\ z' &= z / (z/d) = d \end{aligned}$$

This matches the derivation from similar triangles above, and shows the essential characteristic of perspective projection: the projected x' and y' are inversely proportional to the depth z , producing the visual effect of foreshortening (farther objects appear smaller).

Q5. Derive the expression and matrix representation for Oblique Parallel Projection. [8 Marks]

In Simple English

In an oblique projection, the 'light rays' hit the flat viewing plane at a slanted angle (not straight-on), which lets us see both the front face of an object and a bit of its side/top in a single flat picture — useful for technical drawings.

Oblique Parallel Projection

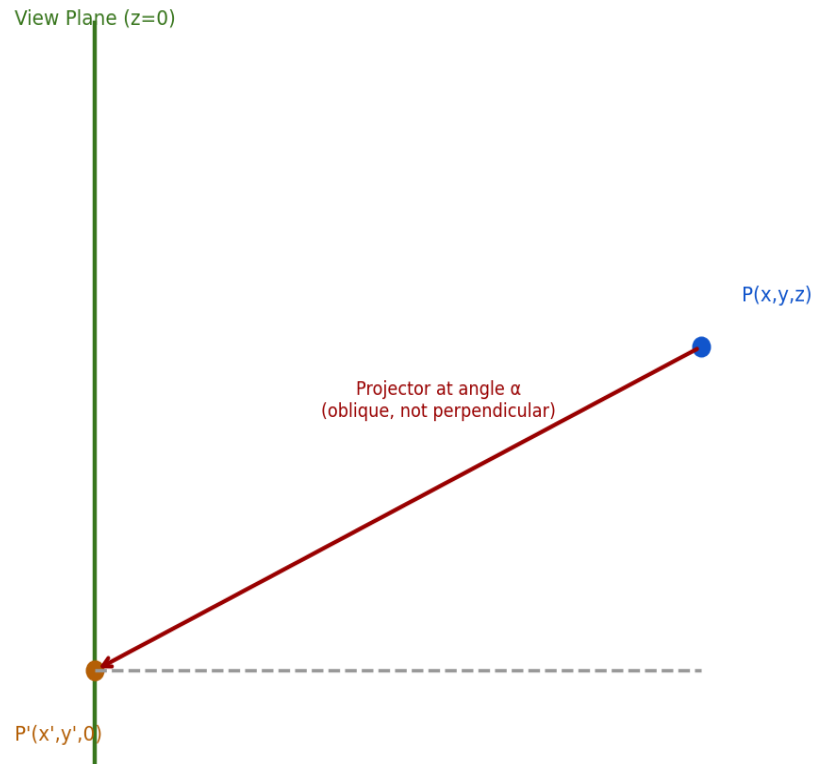


Figure: Oblique Parallel Projection

Derivation

Consider a point $P(x, y, z)$ projected obliquely onto the view plane $z=0$. The projector makes an angle α with the view plane, and its direction (as seen looking along the z -axis) makes an angle ϕ with the x -axis. Let L be the length of the projected line from $(x, y, 0)$ to the final projected point $P'(x', y', 0)$:

$$L = z / \tan \alpha \quad (\text{from the right-triangle formed by } z, L, \text{ and angle } \alpha)$$

The projected point's coordinates are then the original (x, y) shifted by this length L in direction ϕ :

$$x' = x + L \cdot \cos \phi = x + z \cdot (\cos \phi / \tan \alpha)$$

$$y' = y + L \cdot \sin \phi = y + z \cdot (\sin \phi / \tan \alpha)$$

Defining $L1 = 1/\tan \alpha = \cot \alpha$, we can write:

$$x' = x + z \cdot L1 \cdot \cos \phi$$

$$y' = y + z \cdot L1 \cdot \sin \phi$$

$$z' = 0$$

Matrix Representation

$$[Poblique] = \begin{vmatrix} 1 & 0 & L1 \cdot \cos\phi & 0 \\ 0 & 1 & L1 \cdot \sin\phi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Special cases: when $\alpha=45^\circ$ (so $L1=1$), this is called Cavalier projection (depth lines retain their true length). When $\alpha=\arctan(2)$ ($\approx 63.4^\circ$, so $L1=0.5$), this is called Cabinet projection (depth lines are foreshortened to half their true length, producing a more realistic look).

Q6 & Q7. Define polygon rendering methods and the term Rendering. [8 Marks]

In Simple English

Rendering is the whole process of turning a 3D scene description into the final realistic-looking 2D picture, including working out lighting, colors, and shadows.

Technical Explanation

Rendering: The process of generating a photorealistic or stylized 2D image from a 3D scene description, by simulating the effects of geometry, viewpoint, texture, lighting, and shading.

Polygon Rendering Methods

- **Constant-Intensity (Flat) Shading:** Computes a single intensity value for an entire polygon face, assuming a single normal and light source direction. Fast but produces a faceted look.
- **Gouraud Shading (Intensity Interpolation):** Computes intensities at each vertex, then linearly interpolates these across the polygon's interior, producing smoother transitions between faces.
- **Phong Shading (Normal-Vector Interpolation):** Interpolates the surface normal vector itself across the polygon, and computes the lighting equation at every individual pixel — producing the most realistic highlights and shading.

Q8. Define lighting model and ambient light. [8 Marks]

In Simple English

A lighting model is the math formula used to figure out how bright and what color each point on a surface should look, based on the lights around it. Ambient light is the general background light that reaches every surface evenly, even parts not directly facing any light — like the soft light in a room with no direct sunlight.

Technical Explanation

Illumination (Lighting) Model: A mathematical formula used to calculate the intensity and color of light reflected from a point on an object's surface, based on the surface's material properties, its orientation (normal vector), and the position/properties of light source(s) in the scene.

Ambient Light: A simplified approximation of indirect, non-directional background illumination that results from light bouncing repeatedly off multiple surfaces in the environment. It is modeled as a constant intensity term ($I_a \cdot k_a$) that is added uniformly to every surface, regardless of its orientation or position relative to the actual light sources — ensuring that no part of a scene is left completely black.

Q9, Q10 & Q11. Differentiate / Explain Gouraud shading and Phong shading. How is Phong shading better? [8 Marks]

In Simple English

Gouraud shading works out the brightness only at the corners of each triangle and blends the colors smoothly in between. Phong shading is more careful — it works out the actual surface direction at every single pixel and calculates lighting there directly, which makes shiny highlights look much more realistic.

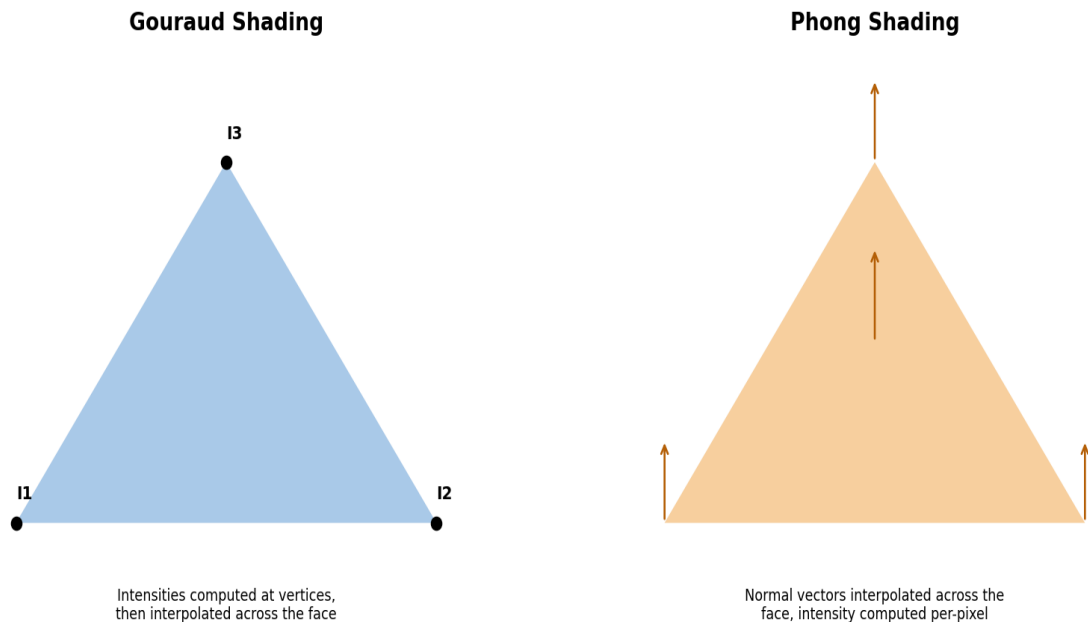


Figure: Gouraud Shading vs Phong Shading

Gouraud Shading (Intensity Interpolation)

11. Compute the average surface normal at each polygon vertex (by averaging the normals of all polygons sharing that vertex).
12. Apply an illumination model at each vertex to compute its intensity.
13. Linearly interpolate these vertex intensities across the surface of the polygon (along edges, then across scan lines).

Phong Shading (Normal-Vector Interpolation)

14. Compute the average surface normal at each vertex, exactly as in Gouraud shading.
15. Linearly interpolate the normal vector itself across the polygon's surface (along edges, then scan lines) — not just the final intensity.
16. Apply the illumination model at every individual pixel, using its interpolated normal, to compute the pixel's own intensity directly.

Why Phong Shading Is Better

- Because lighting is calculated per-pixel (using the true interpolated normal) rather than per-vertex, Phong shading can accurately reproduce sharp specular highlights that fall in the middle of a polygon face — something Gouraud shading often misses or blurs, since it only samples intensity at the vertices.
- Phong shading produces noticeably smoother, more realistic curved-surface appearance, especially under point light sources.
- The trade-off is computational cost: Phong shading requires far more calculations per pixel (a full lighting computation for every pixel) compared to Gouraud's simple linear interpolation of pre-computed intensities.

Aspect	Gouraud Shading	Phong Shading
Interpolates	Intensity values	Surface normal vectors
Lighting computed	Once per vertex	Once per pixel
Specular highlights	Often inaccurate/missed	Accurate and sharp
Computational cost	Lower	Higher

Q12. Explain the importance of shading methods. [8 Marks]

In Simple English

Shading is what makes a flat set of triangles actually look like a solid, rounded, realistic 3D object instead of a bunch of obviously flat panels.

Importance

- Provides visual cues about the 3D shape and curvature of an object, which flat, unshaded polygons cannot convey
- Removes the artificial 'faceted' appearance of polygon meshes, producing smoother, more natural-looking curved surfaces
- Simulates how light interacts with different materials, contributing significantly to photorealism in rendered images
- Helps convey depth and spatial relationships between overlapping objects in a scene
- Essential for applications ranging from games and films to CAD visualization and scientific rendering

Q13. Why is it necessary to remove hidden surfaces? Explain the Z-Buffer algorithm in detail.

[8 Marks]

In Simple English

When you look at a 3D scene from one viewpoint, some objects block your view of others behind them — just like in real life. A computer must figure out and remove (hide) those blocked parts so the picture looks correct, instead of showing everything, including things that should be invisible.

Why Hidden-Surface Removal Is Necessary

- Without it, surfaces that should be blocked by closer objects would incorrectly show through, producing a confusing, unrealistic image (like an X-ray)
- Correct depth ordering is essential for realistic, physically-plausible rendering
- It also improves rendering efficiency, since surfaces determined to be hidden need not be shaded/colored at all in some algorithms

Z-Buffer (Depth-Buffer) Algorithm

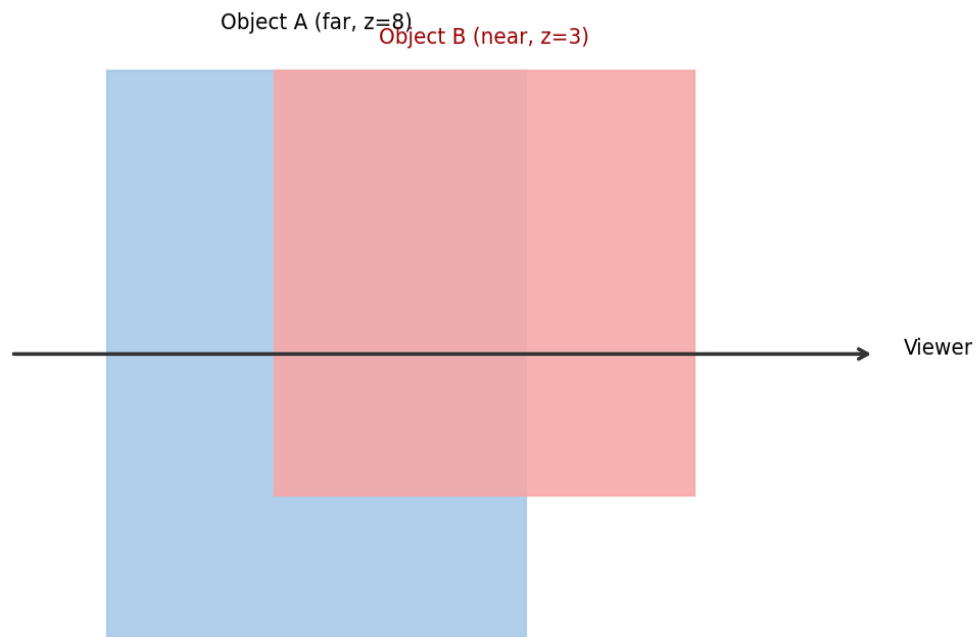
The Z-buffer method is an image-space algorithm that compares surface depths (z-values) at each pixel position. It uses two buffers of equal size to the frame buffer:

- **Depth Buffer (Z-buffer):** Stores the smallest (closest) z-value encountered so far, for each pixel — initialized to the maximum possible depth (farthest away).
- **Frame Buffer:** Stores the color/intensity value corresponding to the closest surface found so far at that pixel — initialized to the background color.

Algorithm Steps

17. Initialize the depth buffer to the maximum depth value, and the frame buffer to the background color, at every pixel position.
18. For each polygon in the scene, compute the depth (z-value) at every (x,y) pixel position it covers.
19. Compare this computed depth to the current value stored in the depth buffer at that pixel: if the new depth is smaller (i.e., closer to the viewer), update both the depth buffer (with the new z) and the frame buffer (with the new color/intensity); otherwise, discard it (leave existing values unchanged, since a closer surface already occupies that pixel).
20. After all polygons have been processed, the frame buffer contains the correctly visible (hidden-surface-removed) image.

Z-Buffer (Depth-Buffer) Hidden Surface Removal



*For each pixel, keep the smallest z (closest) value seen so far
and only draw that surface's color — hides farther surfaces automatically*

Figure: Z-Buffer Hidden Surface Removal

This algorithm is simple to implement, works on unsorted polygon lists, and is widely implemented directly in graphics hardware (GPUs) for real-time rendering.

Q14. How are geometric tables used to represent a 3D object? Give conditions to generate an error-free table. [8 Marks]

In Simple English

A 3D object made of flat faces (polygons) can be fully described using simple lists: a list of corner points, a list of which points form each edge, and a list of which edges form each face.

Technical Explanation

A polygon (polyhedron) surface is commonly represented using three geometric tables that together define its geometry and connectivity:

- **Vertex Table:** Stores the (x, y, z) coordinate values for each vertex in the object.
- **Edge Table:** Stores pointers back into the vertex table, indicating which pair of vertices form each edge.
- **Polygon (Surface/Face) Table:** Stores pointers back into the edge table, indicating which edges bound each polygon face.

Example: for a cube, the vertex table lists its 8 corner coordinates, the edge table lists the 12 edges (each referencing 2 vertices), and the polygon table lists the 6 faces (each referencing 4 edges).

Conditions for an Error-Free Table

- Every edge listed in the edge table must reference exactly two valid, existing vertices from the vertex table
- Every edge must be shared by exactly two polygon faces (for a closed/solid object) — an edge belonging to only one face indicates a hole or open surface
- Consistent vertex ordering (e.g., always counter-clockwise when viewed from outside) must be maintained for every polygon, so that surface normals can be computed consistently for correct shading and back-face culling
- No duplicate vertices or degenerate (zero-length) edges should exist in the tables

Maintaining these geometric tables (rather than a simple unstructured list of independent polygons) also allows consistency checks — such as verifying every edge connects exactly two faces — to automatically detect errors like missing faces or gaps in the model.

Q15. What is a Bezier curve? Explain the various methods for generating non-planar surfaces.

[8 Marks]

In Simple English

A Bezier curve is a smooth curve that is bent and shaped by a small number of 'control points' that act like magnets pulling the curve toward them, without the curve necessarily touching most of them.

Technical Explanation — Bezier Curve

A Bezier curve is a parametric curve defined by a set of $n+1$ control points $P_0, P_1, \dots, P_n$, using Bernstein polynomials as blending functions:

$$P(t) = \sum_{k=0}^{n} [P_k \cdot \text{BEZ}_{k,n}(t)], \quad 0 \leq t \leq 1$$

where the Bernstein blending function is:

$$\text{BEZ}_{k,n}(t) = C(n,k) \cdot t^k \cdot (1-t)^{(n-k)}$$

Key Properties

- The curve always passes exactly through the first and last control points (P_0 and P_n), but generally not through the intermediate ones
- The curve lies entirely within the convex hull of its control points
- The degree of the curve is one less than the number of control points ($n+1$ points give a curve of degree n)
- The curve's shape can be intuitively modified simply by dragging its control points

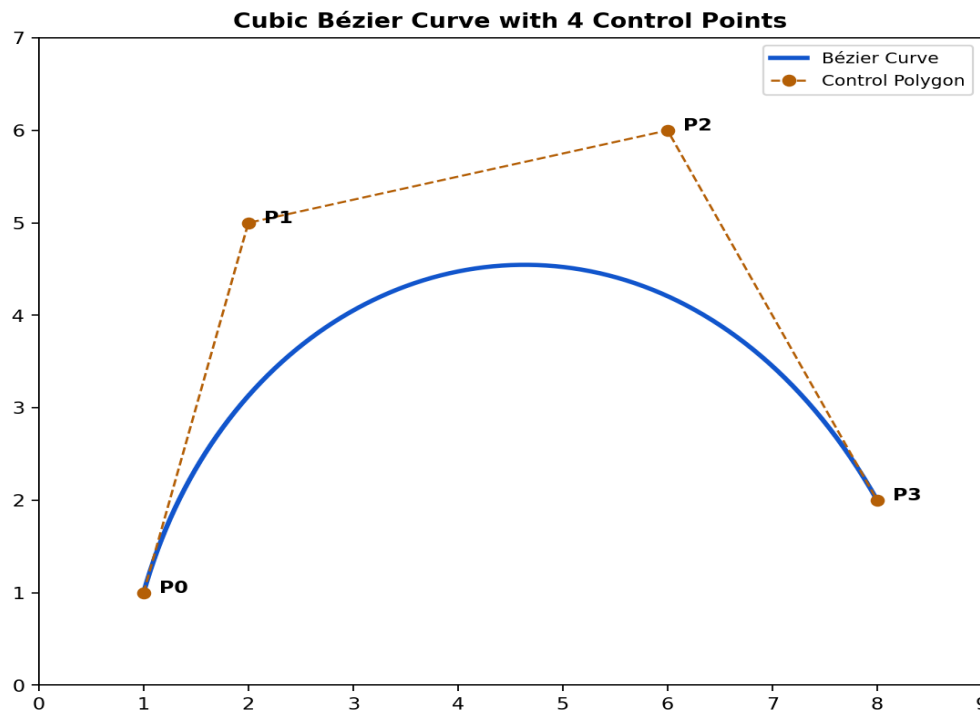


Figure: A Cubic Bézier Curve Defined by 4 Control Points

Methods for Generating Non-Planar (Curved) Surfaces

- **Bezier Surfaces:** Formed by taking the Cartesian product of two Bezier curves (in two parametric directions u and v) over a rectangular grid (net) of control points, producing a smoothly curved patch.
- **B-Spline Surfaces:** Similar to Bezier surfaces but built from B-spline basis functions, giving more local control — moving one control point only affects a nearby region of the surface, rather than the entire patch.
- **NURBS (Non-Uniform Rational B-Splines):** An extension of B-splines using rational (weighted) control points, capable of exactly representing both free-form curved surfaces and precise conic shapes (spheres, cylinders); the industry standard in CAD and animation software.
- **Quadric Surfaces:** Surfaces described directly by second-degree (quadratic) equations, such as spheres, ellipsoids, and cylinders.
- **Sweep Representations:** A 2D curve or shape is swept along a specified 3D trajectory (translational or rotational) to generate a surface, e.g., generating a surface of revolution by rotating a curve about an axis.

7. Multimedia Technology & Authoring

Q1. What is / Define Multimedia Authoring. [8 Marks]

In Simple English

Multimedia authoring is the process of combining text, pictures, sound, and video together into one finished interactive product, like a training CD, an e-learning course, or an interactive kiosk.

Technical Explanation

Multimedia authoring is the process of creating multimedia productions/applications by assembling and sequencing different media elements — text, images, audio, video, and animation — often including interactivity and navigational logic, using specialized authoring software (an authoring tool/system).

Q2. What is multimedia technology? [8 Marks]

In Simple English

Multimedia technology is all the hardware and software that lets computers combine and present multiple types of media — text, images, sound, video, and animation — together at once.

Technical Explanation

Multimedia technology refers to the computer-based hardware (sound cards, graphics cards, storage/optical drives, input/output devices) and software (compression codecs, authoring tools, media players) used to capture, process, store, and present combined content across multiple media formats in an integrated, often interactive, manner.

Q3. Explain the Multimedia Team with their roles and skills. [8 Marks]

In Simple English

Building a multimedia product needs many different specialists working together — writers, artists, programmers, and managers — each contributing a different skill.

Technical Explanation — Typical Multimedia Team Roles

- **Project Manager:** Oversees the entire project — budget, schedule, coordination between team members, and client communication.
- **Multimedia Designer:** Designs the overall look, feel, layout, and interaction flow of the product.
- **Writer / Content Specialist:** Writes scripts, narration, on-screen text, and educational content.
- **Graphic Artist / Illustrator:** Creates visual assets — illustrations, icons, backgrounds, and UI graphics.
- **Audio/Video Specialist:** Records, edits, and produces sound effects, music, narration, and video clips.
- **Animator:** Creates 2D/3D animations and motion graphics for the product.
- **Multimedia Programmer:** Writes the code/scripting logic that ties all media elements together and implements interactivity.
- **Quality Assurance (Tester):** Tests the final product across different devices/platforms for bugs, usability, and performance.

Q4, Q5 & Q8. List/Classify different multimedia authoring tools and explain any one/two. [8 Marks]

In Simple English

Authoring tools are the software programs used to actually build a multimedia project — some work like a slideshow, some like a flowchart, and some like a timeline (similar to a movie editor).

Types of Multimedia Authoring Tools

- **Card/Page-Based Tools:** Organize content as a stack of virtual cards or pages that can be linked together (e.g., early HyperCard, some e-learning tools). Good for content that is naturally organized into discrete screens.
- **Icon/Flowchart-Based Tools:** Multimedia elements and interaction logic are organized visually as icons connected in a flowchart, representing the flow of the program (e.g., Authorware). Intuitive for designing branching, decision-based interactivity.
- **Time-Based Tools:** Elements are arranged along a timeline with multiple tracks (similar to a video editor), useful for animations and presentations with a fixed temporal sequence (e.g., Adobe Animate/Director).
- **Object-Oriented / Scripting Tools:** Provide a full scripting language for maximum flexibility, treating each media element as a programmable object (e.g., Adobe Flash's ActionScript, or general-purpose programming with multimedia libraries).

Example Explained: Time-Based Authoring Tool

A time-based tool (such as Adobe Animate) organizes a project along a timeline divided into frames, with multiple layers/tracks for different media elements (graphics, audio, video). Objects can be animated by defining keyframes at specific points in time, with the software interpolating (tweening) the motion in between — making it ideal for producing animations, cartoons, and time-synchronized presentations.

Example Explained: Icon/Flowchart-Based Tool

An icon-based tool represents each interactive element (a piece of text, a question, a branch, a media clip) as an icon placed onto a flowline. The author drags icons into place and connects them to define the logical sequence and interactivity (e.g., 'if the user answers correctly, branch to icon A; otherwise branch to icon B') — making it especially popular for building interactive tutorials and training courseware.

Q6. Define Multimedia and Multimedia Product. [8 Marks]

In Simple English

Multimedia just means using more than one type of media (text + images + sound + video) together. A multimedia product is the finished thing built using them, like an interactive CD-ROM, a mobile app, or a website.

Technical Explanation

Multimedia: The integrated use of multiple forms of media content — text, audio, still images, animation, video — combined and often made interactive through a computer, to inform or entertain the user.

Multimedia Product: A finished, deliverable application or content package that integrates multiple media types and typically supports user interactivity, such as an e-learning module, interactive kiosk software, a video game, or a multimedia website.

Q7. What is Multimedia Product Development? Explain elements of multimedia with examples. [8 Marks]

In Simple English

Multimedia product development is the whole journey of planning, designing, building, and testing a multimedia project from an idea to a finished product.

Technical Explanation — Development Process

21. Planning & Analysis: Define objectives, target audience, budget, and required content.
22. Design: Create storyboards, flowcharts, and prototypes of screen layouts and interaction flow.
23. Content Production: Produce/collect the actual text, graphics, audio, and video assets.
24. Integration/Authoring: Combine all media elements using an authoring tool, adding interactivity.
25. Testing: Verify functionality, usability, and performance across target platforms.
26. Delivery/Distribution: Package and distribute the final product (CD/DVD, web, app store).

Elements of Multimedia

- **Text:** The most basic element, used for titles, labels, and detailed content (e.g., an article body).
- **Graphics/Images:** Still visual elements — photographs, illustrations, icons — used to convey information visually.
- **Audio:** Sound elements — music, narration, sound effects — that add emotional and informational depth.
- **Video:** Moving visual sequences (often combined with audio) — e.g., recorded footage, tutorials.
- **Animation:** Computer-generated moving graphics, such as animated diagrams or cartoon characters.
- **Interactivity:** Allows the user to control the flow/content, e.g., clicking buttons, answering quizzes, navigating menus.

Q9. Write down the features of image editing tools. [8 Marks]

In Simple English

Image editing software (like Photoshop or GIMP) gives you tools to fix, retouch, and creatively alter photographs and graphics.

Key Features

- Layer-based editing, allowing independent, non-destructive manipulation of different image elements
- Selection tools (lasso, magic wand, marquee) for isolating specific parts of an image
- Color correction and adjustment tools (brightness, contrast, hue/saturation, curves, levels)
- Filters and effects (blur, sharpen, distortion, artistic filters)
- Support for multiple file formats (JPEG, PNG, GIF, TIFF, PSD, etc.) and various color modes (RGB, CMYK, grayscale)
- Retouching tools (clone stamp, healing brush) for removing blemishes/unwanted objects
- Text and vector-drawing tools for adding typography and shapes
- Support for image compositing (combining multiple images) and masking

8. Color Models, Typography & Formats

Q1. Explain RGB and CMYK color model. [8 Marks]

In Simple English

RGB is how screens make color by mixing colored light. CMYK is how printers make color by mixing colored ink on white paper.

Technical Explanation

RGB Color Model: An additive color model that produces colors by combining varying intensities of Red, Green, and Blue light. It is represented as a 3D cube where each axis represents one primary color's intensity (0 to 255, or 0 to 1). Combining all three at full intensity produces white; combining none produces black. Used for screens, monitors, cameras, and any light-emitting display.

CMYK Color Model: A subtractive color model used in color printing, based on Cyan, Magenta, Yellow, and Key (black) inks. Each ink absorbs (subtracts) certain wavelengths of light reflecting off the white paper beneath it; combining all three chromatic inks theoretically produces black, though a separate true-black (K) ink is added for deeper blacks and to reduce ink costs.

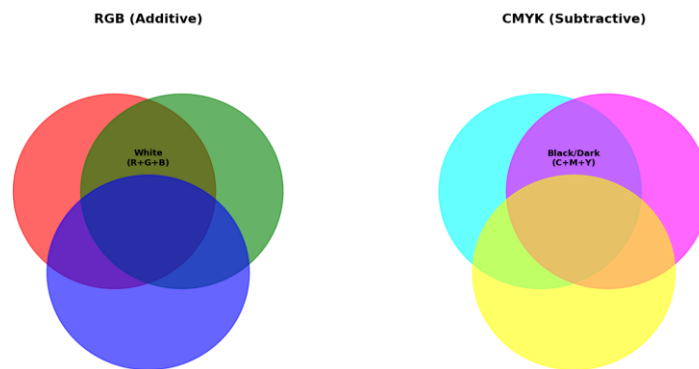


Figure: RGB (Additive) vs CMYK (Subtractive) Color Models

Q2 & Q3. Why is CMYK called a subtractive color model, and RGB an additive one? [8 Marks]

In Simple English

A screen starts pitch black and adds light to create colors — that's why RGB is 'additive'. A printed page starts white and each layer of ink blocks (subtracts) some of that light from being reflected back to your eyes — that's why CMYK is 'subtractive'.

Technical Explanation

RGB is Additive: A display starts from black (no light emitted) and adds Red, Green, and Blue light in varying amounts; the more light added, the brighter/whiter the result becomes. This models how light itself combines.

CMYK is Subtractive: Printing starts from a white surface (paper, which reflects all wavelengths of light) and each ink layer absorbs (subtracts) specific wavelengths from the white light before it reflects back to the eye; the more ink layers combined, the darker/less light is reflected. This models how pigments/inks work rather than light sources.

Aspect	RGB (Additive)	CMYK (Subtractive)
Starting Point	Black (no light)	White (paper reflecting all light)
Operation	Adds colored light	Removes/absorbs wavelengths of light
All Combined	White	Black (theoretically)
Used For	Screens, displays, digital media	Printing, publishing

Q4. Explain the terms Jaggies, Bitmap, TrueType, and Fontographer. [8 Marks]

In Simple English

These are all terms related to how text and images are drawn on screen. Jaggies are the stair-step edges you sometimes see on diagonal lines. Bitmap fonts/images are stored as fixed dot patterns. TrueType fonts are drawn using flexible math outlines so they scale cleanly. Fontographer is a tool used to design fonts.

Technical Explanation

Jaggies: The visible staircase-like distortion (aliasing) that appears along diagonal or curved edges when they are represented on a discrete pixel grid at insufficient resolution. Reduced through anti-aliasing techniques.

Bitmap: An image or font representation stored as a fixed grid (raster) of individual pixel values. Bitmap images/fonts do not scale well — enlarging them makes the individual pixels (and jaggies) visibly larger.

TrueType: An outline (vector) font format developed by Apple and Microsoft, where each character is defined mathematically using curves (quadratic Bézier splines) rather than fixed pixels — allowing the font to be scaled to any size while remaining crisp and smooth.

Fontographer: A specialized software application used by type designers to create and edit outline fonts (such as TrueType and PostScript Type 1 fonts), letting designers draw and adjust letterforms and generate usable font files.

9. Animation, Video & Audio Technologies

Q1 & Q2. What is Animation and Video? [8 Marks]

In Simple English

Animation is the trick of showing a fast sequence of slightly-different still pictures so our eyes perceive smooth movement. Video is a recording of real, continuously moving footage, captured by a camera, made of a sequence of individual frames played back quickly.

Technical Explanation

Animation: The technique of creating an illusion of motion by rapidly displaying a sequence of individually-created static images (frames), each slightly different from the last, exploiting the persistence of vision of the human eye.

Video: A sequence of captured (recorded) or generated images (frames) displayed in rapid succession to represent moving visual scenes, typically synchronized with audio, and usually stored/transmitted using compression techniques (codecs).

Q3. Explain the principles of Animation for today's development. [8 Marks]

In Simple English

These are classic rules (originally from Disney animators) that make animated movement feel believable and alive, and they are still followed today in games, films, and UI motion design.

Key Principles

- **Squash and Stretch:** Deforming an object slightly during motion (e.g., a bouncing ball flattening on impact) conveys weight and flexibility.
- **Anticipation:** A small preparatory movement before the main action, helping viewers anticipate what's about to happen.
- **Staging:** Presenting an action clearly so the viewer's attention is drawn to the most important part of the scene.
- **Straight-Ahead & Pose-to-Pose:** Two different animation workflows — drawing frame-by-frame in sequence, versus defining key poses first and filling in between them.
- **Follow-Through & Overlapping Action:** Parts of a body/object continue moving briefly after the main motion stops (e.g., hair or clothing trailing behind).
- **Slow In and Slow Out (Easing):** Objects accelerate and decelerate gradually rather than moving at constant speed, mimicking real-world physics.
- **Arcs:** Natural motion tends to follow curved paths (arcs) rather than straight lines.
- **Timing:** The number of frames used for an action determines its perceived speed and weight.
- **Exaggeration:** Slightly amplifying actions/expressions to make them more visually appealing and readable.
- **Appeal:** Designing characters/motion that are visually interesting and engaging to the audience.

Today, these same principles are directly applied in modern 3D animation, game character motion, and even UI/UX 'micro-interaction' animations on websites and mobile apps.

Q4. Explain the Types of Animation. [8 Marks]

- **2D (Traditional / Cel) Animation:** Hand-drawn or vector-based frame-by-frame animation on a flat 2D plane, e.g., classic cartoons.
- **3D (Computer) Animation:** Animating 3D models (rigged with skeletons/bones) within 3D software, rendered with lighting and shading, e.g., Pixar films.
- **Motion Graphics:** Animated graphic design elements (text, shapes, logos) commonly used in title sequences, advertisements, explainer videos.
- **Stop-Motion Animation:** Physical objects/models are photographed frame-by-frame with small movements between shots, then played back in sequence, e.g., claymation.
- **Keyframe / Procedural Animation:** Animators define important 'key' poses (keyframes), with software interpolating (tweening) the motion in between; procedural animation instead uses algorithms/physics to generate motion automatically.
- **Morphing:** Smoothly transforming one image/shape into another over time.

Q5. Describe major broadcasting video standards. [8 Marks]

In Simple English

Different countries historically adopted different technical formats for broadcasting TV signals — these are called video/broadcasting standards.

Technical Explanation

Standard	Full Form	Frame Rate	Lines	Regions
NTSC	National Television System Committee	~30 fps (29.97)	525 lines	USA, Canada, Japan
PAL	Phase Alternating Line	25 fps	625 lines	Europe, Asia, Australia
SECAM	Séquentiel Couleur à Mémoire	25 fps	625 lines	France, Russia, parts of Africa

Modern digital broadcasting standards have largely superseded these analog standards, including ATSC (USA), DVB (Europe), and ISDB (Japan), which support high-definition resolutions and digital compression (MPEG).

Q6 & Q7. What are video encoding techniques? How does video work, and what are the compression techniques? *[8 Marks]*

In Simple English

A raw video file is enormous, so we use clever compression tricks to shrink it down to a manageable size, while trying to keep it looking as close to the original as possible.

How Video Works

A digital video is fundamentally a sequence of still image frames (a 'frame rate' such as 24, 30, or 60 frames per second) displayed in rapid succession, typically synchronized with an audio track. Because uncompressed video data is extremely large, video encoding (compression) is used to reduce file size for storage and transmission.

Video Compression Techniques

- **Spatial (Intra-frame) Compression:** Compresses each frame independently by removing redundant information within that single image (similar to JPEG image compression), exploiting the fact that neighboring pixels are often similar.
- **Temporal (Inter-frame) Compression:** Exploits redundancy between consecutive frames, storing only the differences (changes) from a previous reference frame rather than storing every frame in full — dramatically reducing size since most video scenes change little between adjacent frames.
- **Lossy Compression:** Permanently discards some visual information that is less perceptible to the human eye, achieving very high compression ratios (used by most codecs like H.264, H.265, MPEG).
- **Lossless Compression:** Compresses without discarding any data, allowing perfect reconstruction, but achieves much lower compression ratios.

Common video encoding standards (codecs) include MPEG-2, H.264/AVC, H.265/HEVC, and VP9 — each combining spatial and temporal compression along with entropy coding to minimize file size while preserving acceptable visual quality.

Q8. What is MIDI? Explain the MIDI hardware, software, and communication between them.

[8 Marks]

In Simple English

MIDI doesn't record actual sound — it records instructions, like sheet music in digital form (which note, how hard it was pressed, how long) so that an electronic instrument or computer can play it back using its own sounds.

Technical Explanation

MIDI (Musical Instrument Digital Interface) is a technical standard that describes a communication protocol, digital interface, and set of electrical connectors that allows electronic musical instruments, computers, and other audio devices to connect and communicate performance data with one another.

MIDI Hardware

- **MIDI Controller/Keyboard:** An input device (often a piano-style keyboard) that generates MIDI messages when played.
- **MIDI Interface/Port:** 5-pin DIN connectors (IN, OUT, THRU) or modern USB-MIDI interfaces that physically carry the MIDI signal between devices.
- **Sound Module/Synthesizer:** Hardware that receives MIDI messages and generates actual audio output based on them.

MIDI Software

- **Sequencer Software:** Records, edits, and plays back MIDI data (e.g., Cakewalk, Cubase, DAWs like Ableton Live).
- **Virtual Instruments (Soft-Synths):** Software that interprets MIDI messages and synthesizes sound digitally, without dedicated external hardware.

Communication Between Hardware and Software

When a key is pressed on a MIDI controller, it generates a MIDI message (e.g., Note On, note number, velocity) which is transmitted serially through the MIDI cable/USB connection to a computer or sound module. The receiving sequencer software or synthesizer interprets these messages and either records them for later playback/editing, or immediately generates the corresponding audio output.

Q9. Define Red Book standard. [8 Marks]

In Simple English

The Red Book is simply the original official rulebook that defines exactly how music is stored on a standard audio CD.

Technical Explanation

The Red Book (formally IEC 60908) is the technical standard that defines the format of audio compact discs (CD-DA — Compact Disc Digital Audio), specifying details such as the sampling rate (44.1 kHz), bit depth (16-bit), stereo channel encoding, and the physical/logical layout of data on the disc, ensuring all Red Book-compliant CDs can be played on any standard CD player.

Q10. Differentiate between MIDI and Digital Audio. [8 Marks]

In Simple English

MIDI is like sheet music — it just stores instructions on what notes to play. Digital audio is like an actual audio recording — it stores the real sound waveform itself.

Aspect	MIDI	Digital Audio
What it Stores	Performance instructions (note, velocity, duration, instrument)	The actual recorded sound waveform (sampled amplitude values)
File Size	Very small (just instructions)	Much larger (stores actual audio data)
Sound Source	Depends entirely on the playback synthesizer/sound module	Sound is fixed/embedded in the recording itself
Editability	Easily edited (change notes, instruments, tempo)	Harder to edit (must directly process the waveform)
Examples	.mid files	WAV, MP3, FLAC files

10. Short Notes

The exam typically asks students to attempt 'any two' short notes from this list. Since a complete answer set is needed, a concise note is provided below for every topic listed; where a topic duplicates an earlier full answer, a cross-reference is given alongside a brief summary.

1. Graphics and Image Processing [5 Marks]

See Section 1, Q4 for the full comparison. In brief: Computer graphics synthesizes new images from geometric/mathematical descriptions (model → image), whereas image processing takes an existing image (like a photograph) and enhances, restores, or analyzes it (image → image/description). Graphics is generative; image processing is analytical/corrective.

2. Video Controller [5 Marks]

In Simple English

The video controller is the hardware 'manager' that constantly reads the picture data from memory and sends it to the screen, refreshing it many times every second.

Technical Explanation

A video controller (display controller) is a hardware component that controls the operation of a raster-scan display device. It retrieves pixel intensity values from the frame buffer and generates the horizontal and vertical synchronization signals needed to refresh the screen at a set rate (e.g., 60-75 Hz), converting stored digital pixel data into the analog/digital signal driving the monitor. It typically contains its own dedicated memory access circuitry to independently scan through the frame buffer without burdening the CPU.

3. Visualization in Computer Graphics [5 Marks]

Visualization is the use of computer-generated imagery to represent abstract or large-scale numerical/scientific data visually, making trends, patterns, and relationships easier to identify than in raw tabular data. See Section 1, Q6 for full details, including scientific, business, and medical visualization examples.

4. Visualization of Data Set [5 Marks]

In Simple English

This means taking a big pile of numbers (a dataset) and turning it into pictures — like graphs or 3D models — so people can understand it quickly.

Technical Explanation

Visualization of a data set involves mapping data attributes (numerical or categorical values) onto visual properties such as position, size, color, or shape, in order to reveal structure, trends, outliers, and relationships within the data. Techniques range from simple 2D charts (bar, line, scatter plots) to complex 3D volumetric rendering (e.g., visualizing medical CT scan data as a 3D reconstructed organ) and multidimensional techniques like parallel coordinates for high-dimensional datasets.

5. Two Successive Rotations [5 Marks]

Two successive rotations about the same point/axis are additive — combining rotation by θ_1 followed by θ_2 is mathematically equivalent to a single rotation by $(\theta_1 + \theta_2)$. This is proven by multiplying the two rotation matrices and applying trigonometric angle-sum identities. See Section 4, Q5 & Q6 for the full derivation.

6. Types of Animation [5 Marks]

The major types are: 2D (traditional/cel) animation, 3D (computer) animation, motion graphics, stop-motion animation, keyframe/procedural animation, and morphing. See Section 9, Q4 for a full explanation of each.

7. Animation Techniques [5 Marks]

In Simple English

These are the actual practical methods animators use to create motion, from drawing every frame by hand to letting the computer calculate movement automatically.

Technical Explanation

- **Frame-by-Frame (Traditional) Animation:** Every single frame is drawn individually — labor-intensive but offers full creative control.
- **Keyframing / Tweening:** The animator defines only the important 'key' poses; software automatically interpolates (generates) the in-between frames.
- **Motion Capture (MoCap):** Sensors placed on a real performer's body record actual movement, which is then mapped onto a digital 3D character/skeleton.
- **Rigging & Skeletal Animation:** A digital skeleton ('rig') of bones and joints is attached to a 3D model, allowing the animator to pose/move the model naturally.
- **Procedural Animation:** Motion is generated automatically through algorithms/physics simulation (e.g., cloth or hair swaying realistically) rather than being manually keyframed.

8. Animation Principles [5 Marks]

The 12 classic principles of animation — including squash and stretch, anticipation, staging, follow-through, slow in/slow out, arcs, timing, exaggeration, and appeal — guide animators in producing motion that feels natural and engaging. See Section 9, Q3 for a full list and explanation.

9. Illumination Models [5 Marks]

In Simple English

An illumination model is the formula used to calculate how bright and what color a point on a surface should appear, by combining a few different types of light contribution.

Technical Explanation

An illumination (lighting) model computes the total intensity of light reflected from a surface point as the sum of three components:

$$I = I_a \cdot k_a + I_d \cdot k_d \cdot (N \cdot L) + I_s \cdot k_s \cdot (R \cdot V)^n$$

ambient diffuse specular

- **Ambient Reflection:** A constant background light term, present regardless of surface orientation or direct light visibility.
- **Diffuse Reflection:** Light that scatters equally in all directions from a matte surface, with intensity depending on the angle between the surface normal (N) and the light direction (L) — following Lambert's cosine law.
- **Specular Reflection:** Bright, mirror-like highlights that depend on the angle between the reflected light direction (R) and the viewer's direction (V), controlled by a shininess exponent n (higher n = smaller, sharper highlight).

The Phong illumination model (the equation shown above) is the most widely used model combining all three components to produce realistic-looking shaded surfaces.

10. Image Formats [5 Marks]

In Simple English

Image formats are the different file types used to save pictures, each with trade-offs between quality, file size, and features like transparency or animation.

Format	Compression	Key Feature
BMP	None (uncompressed)	Simple, large file size, exact pixel data
JPEG	Lossy	Small file size, good for photographs, some quality loss
PNG	Lossless	Supports transparency, no quality loss, larger than JPEG
GIF	Lossless (limited 256 colors)	Supports simple animation
TIFF	Lossless (optional)	High quality, used in printing/publishing/scanning
SVG	Vector (not raster)	Scalable without quality loss, ideal for logos/icons

11. LCD vs LED [5 Marks]

In Simple English

Both are flat-screen display types. LCD uses a separate light source (a backlight, usually fluorescent tubes in older models) shining through liquid crystals to make an image. LED is actually a type of LCD that simply uses small, efficient LED lights as its backlight instead.

Aspect	LCD (traditional)	LED
Backlight	Cold Cathode Fluorescent Lamps (CCFL)	Light-Emitting Diodes (LEDs)
Thickness	Thicker	Thinner, lighter
Power Consumption	Higher	Lower, more energy-efficient
Contrast & Color	Standard	Generally better contrast and color accuracy
Lifespan	Shorter	Longer

12. OpenGL *[5 Marks]*

In Simple English

OpenGL is a free, widely-used toolkit (library) that programmers use to tell the graphics card what 2D/3D shapes, colors, and lighting to draw.

Technical Explanation

OpenGL (Open Graphics Library) is a cross-platform, cross-language application programming interface (API) for rendering 2D and 3D vector graphics, providing a standardized set of functions to communicate directly with the GPU for tasks such as defining geometric primitives, applying transformations, texturing, and lighting/shading calculations. It is widely used in CAD software, games, scientific visualization, and virtual reality applications, and forms the basis for other graphics APIs (such as OpenGL ES for mobile/embedded devices).

13. Video Broadcasting Service *[5 Marks]*

Video broadcasting refers to the transmission of video/audio content to a wide audience using standards such as NTSC, PAL, and SECAM (analog) or ATSC, DVB, and ISDB (digital). See Section 9, Q5 for a full comparison table.

14. Color Palette *[5 Marks]*

In Simple English

A color palette is simply the limited set of colors that an image or display is allowed to use at one time, out of all the colors that could theoretically be shown.

Technical Explanation

A color palette (or color lookup table, CLUT) is an indexed table of a limited number of specific color values selected from a larger color space (e.g., 256 colors chosen from the full 16.7-million-color RGB space). Instead of storing a full color value for every pixel, each pixel simply stores an index/pointer into this palette table, significantly reducing the memory required to store an image — commonly used in formats like GIF and older 8-bit display hardware.

15. MIDI vs Digital Audio *[5 Marks]*

MIDI stores performance instructions (which note, how loud, how long) rather than actual sound, resulting in very small file sizes but requiring a synthesizer to actually produce audio. Digital audio stores the real sampled sound waveform itself, producing much larger files but preserving the exact recorded sound. See Section 9, Q10 for the full comparison table.